

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра Вычислительной техники
(полное название кафедры)

Утверждаю

Зав. кафедрой Якименко А.А.

(подпись, инициалы, фамилия)

«__» _____ 2022 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

Степаненко Владислав Алексеевич

(фамилия, имя, отчество студента – автора работы)

Разработка серверного приложения сервиса дистрибуции музыки

(тема работы)

Автоматики и вычислительной техники

(полное название факультета)

Направление подготовки 09.03.01 Информатика и вычислительная техника

(код и наименование направления подготовки бакалавра)

**Руководитель
от НГТУ**

Романов Евгений Леонидович

(фамилия, имя, отчество)

к.т.н., доцент

(ученая степень, ученое звание)

(подпись, дата)

**Автор выпускной
квалификационной работы**

Степаненко Владислав Алексеевич

(фамилия, имя, отчество)

АВТФ, АВТ-819

(факультет, группа)

(подпись, дата)

Новосибирск 2022

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра Вычислительной техники
(полное название кафедры)

УТВЕРЖДАЮ

Зав. кафедрой Якименко А.А.
(фамилия, имя, отчество)

(подпись, дата)

**ЗАДАНИЕ
НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ БАКАЛАВРА**

студенту Степаненко Владиславу Алексеевичу
(фамилия, имя, отчество)

Направление подготовки 09.03.01 Информатика и вычислительная техника
(код и наименование направления подготовки бакалавра)

Автоматики и вычислительной техники
(полное название факультета)

Тема Разработка серверного приложения сервиса дистрибуции музыки
(полное название темы выпускной квалификационной работы бакалавра)

Исходные данные (или цель работы) Разработка серверного приложения для оригинального сервиса дистрибуции музыки, предоставляющего поиск, хранение, классификацию и стриминг музыки, авторизацию пользователей и сохранение их оценок

Структурные части работы _____

1) Введение

2) Системная аналитика: обзор существующих решений, формирование бизнес-требований к системе, особенности архитектуры системы, моделирование предметной области

3) Выбор средств разработки приложения

4) Архитектурное проектирование: структуры данных, алгоритмы, API

- 5) Функциональное проектирование, структура программного кода
- 6) Тестирование системы, руководство пользователя
- 7) Заключение
- 8) Список литературы
- 9) Приложение
-
-
-
-
-
-
-

Задание согласовано и принято к исполнению.

**Руководитель
от НГТУ**

Студент

Романов Евгений Леонидович

(фамилия, имя, отчество)

к.т.н., доцент

(ученая степень, ученое звание)

(подпись, дата)

Степаненко Владислав Алексеевич

(фамилия, имя, отчество)

АВТФ, АВТ-819

(факультет, группа)

(подпись, дата)

Тема утверждена приказом по НГТУ № _____ от « ____ » _____ 202__ г.
изменена приказом по НГТУ № _____ от « ____ » _____ 202__ г.

ВКР сдана в ГЭК № _____, тема сверена с данными приказа

(подпись секретаря государственной экзаменационной комиссии по защите ВКР, дата)

(фамилия, имя, отчество секретаря государственной
экзаменационной комиссии по защите ВКР)

АННОТАЦИЯ

Выпускная квалификационная работа бакалавра выполнена студентом Степаненко В.А.

Руководитель работы: Романов Е.Л.

Тема: «Разработка серверного приложения сервиса дистрибуции музыки»

Пояснительная записка содержит 88 страниц, 56 рисунков, 1 приложение.

Цель работы: Разработка серверного приложения для оригинального сервиса дистрибуции музыки, предоставляющего поиск, хранение, классификацию и стриминг музыки, авторизацию пользователей и сохранение их оценок.

Ключевые слова: язык программирования Python, фреймворк FastAPI, система ORM SQLAlchemy, система контейнерных приложений Docker, веб-сервер Uvicorn, поисковый движок Elasticsearch.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	7
1 СИСТЕМНАЯ АНАЛИТИКА	8
1.1 Постановка задачи	8
1.2 Обзор существующих решений.....	8
1.2.1 Сравнительная характеристика музыкальных сервисов	8
1.2.2 Особенности музыкальных сервисов	10
1.3 Формирование бизнес-требований.....	11
1.4 Формирование функциональных требований.....	11
1.5 Формирование модели прецедентов	14
1.6 Особенности архитектуры системы и моделирование предметной области.....	15
2 ВЫБОР СРЕДСТВ РАЗРАБОТКИ ПРИЛОЖЕНИЯ	17
2.1 Язык программирования	17
2.2 Фреймворк	17
2.3 СУБД.....	19
2.4 Поисковая система.....	21
2.5 Веб-Сервер.....	24
2.6 Дополнительные средства.....	24
3 АРХИТЕКТУРНОЕ ПРОЕКТИРОВАНИЕ.....	27
3.1 Общая архитектура и взаимодействие компонентов	27
3.2 Структуры данных	28
3.2.1 Объектная модель	28
3.2.2 Структура базы данных	34
3.3 Алгоритмы.....	35
3.3.1 Аутентификация.....	35
3.3.2 Стриминг контента.....	39
3.3.3 Получение рекомендаций на основе пользовательской статистики.....	41
3.4 API.....	43
4 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ, СТРУКТУРА ПРОГРАММНОГО КОДА... ..	45
4.1 Маршрутизация, конечные точки	45
4.2 Токены	45
4.3 CRUD	47
4.3.1 Альбомы, исполнители, треки	47
4.3.2 Очереди	49

4.3.3 Плейлисты.....	50
4.4 Стриминг	51
4.5 Поиск.....	53
5 ТЕСТИРОВАНИЕ СИСТЕМЫ, РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ	57
5.1 Автоматические тесты, Pytest.....	57
5.2 Ручное тестирование, тестирование поиска.....	58
5.2.1 Создание пользователя	59
5.2.2 Вход в систему	59
5.2.3 Обновление изображения пользователя	60
5.2.4 Получение списка случайных треков.....	61
5.2.5 Оценка треков.....	63
5.2.6 Получение рекомендаций к альбому.....	64
5.2.7 Создание плейлиста, добавление песен	64
5.2.8 Получение созданного плейлиста.....	66
5.3 Руководство пользователя	68
ЗАКЛЮЧЕНИЕ.....	74
СПИСОК ЛИТЕРАТУРЫ.....	75
ПРИЛОЖЕНИЯ	78
Приложение А. Описание конечных точек	78

ВВЕДЕНИЕ

Повышение скорости интернета и его общей доступности изменило представление людей о музыке, если раньше прослушивание музыки сопровождалось покупкой альбомов или отдельных песен, то с появлением музыкальных стриминговых сервисов пользователи получили доступ к обширной библиотеке музыки без необходимости выбора конкретных песен и альбомов, а также удобную классификацию и поиск музыки, систему рекомендаций на основе прослушанных произведений и их схожести, возможность сохранять и делиться понравившимися треками, получать всю необходимую информацию о исполнителях и актуальных релизах, а также доступ к этому с использованием удобного им способа: телефона, веб-приложения или приложения для ПК.

Целью данной работы является разработка серверного приложения, предоставляющего функционал стриминга, хранения, редактирования, поиска классифицированной музыки и пользовательской оценки произведений. Для реализации данной цели необходимо выполнить следующее:

1. Провести системную аналитику в области музыкальных стриминговых сервисов;
2. Выбрать подходящие средства для разработки;
3. Выполнить архитектурное проектирование системы;
4. Выполнить функциональное проектирование системы;
5. Протестировать полученную систему и предоставить документацию.

Актуальность работы обуславливается возможностью повышения конкуренции среди сервисов музыкального стриминга на Российском рынке.

1 СИСТЕМНАЯ АНАЛИТИКА

1.1 Постановка задачи

Цель работы – разработать серверное приложение для музыкального стримингового сервиса дистрибуции, хранящее библиотеку треков, альбомов и исполнителей, предоставляющее API для работы с ним.

API приложения должно предоставлять незарегистрировавшимся пользователям возможность поиска и фильтрации произведений, прослушивания треков с использованием стриминга.

API приложения должно предоставлять зарегистрировавшимся пользователям возможности аналогичные незарегистрировавшимся, а также возможность отмечать понравившиеся произведения, управлять очередью, создавать плейлисты и получать рекомендации к прослушиванию.

Пользователи, являющиеся администраторами сервиса, должны иметь возможность пополнять и редактировать библиотеку произведений и сопутствующую информацию при помощи соответствующих функций API.

1.2 Обзор существующих решений

1.2.1 Сравнительная характеристика музыкальных сервисов

Основными используемыми сервисами стриминга музыки в России на данный момент являются Яндекс.Музыка и ВК Музыка, согласно статистике, предоставленной сетью Мегафон на конец 2020 года [1].

Сравнительная характеристика сервисов Яндекс.Музыка, SoundCloud и ВК Музыка приведена в таблице 1.

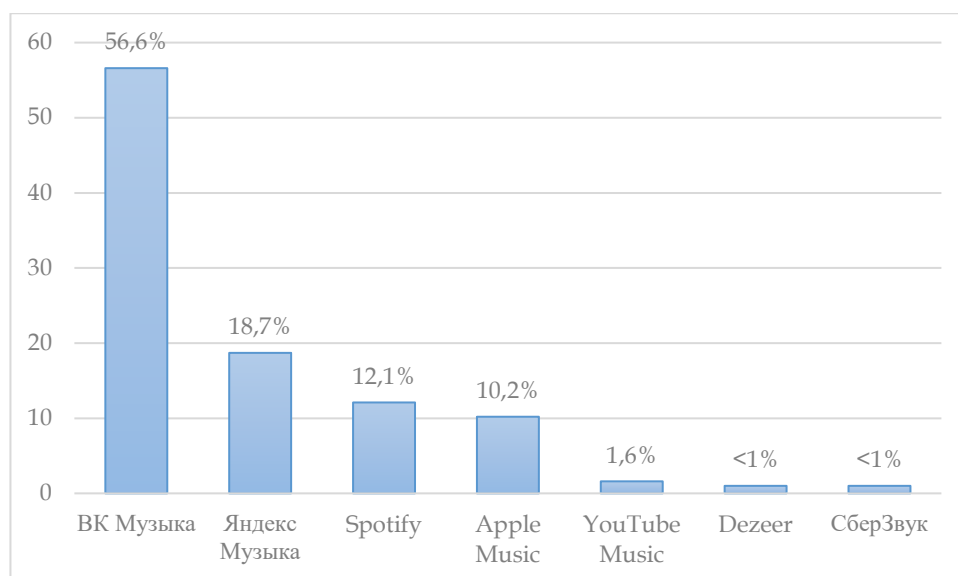


Рисунок 1 — Доли музыкальных сервисов

Таблица 1 — Сравнительная характеристика стриминговых сервисов

	Яндекс.Музыка	SoundCloud	ВК Музыка
1	2	3	4
Устройства	Веб, Windows, Android, IOS	Веб, Windows, Android, IOS	Веб, IOS, Android
Максимальный битрейт \ формат	192 Кбит/с, mp3 или 320 Кбит/с, mp3 (подписка)	128 Кбит\с, MP3 или 256 Кбит\с, AAC (подписка)	Варьируемый битрейт, mp3
Классификация по:			
альбомам	+	+	+-
исполнителям	+	+	+-
жанрам	+	+	+-
Очередь треков:			
управление очередью	+	+	+
синхронизация очереди	+	-	-
Оценивание музыки («лайки»):			
исполнителям	+	-	+
трекам	+	+	+
альбомам	+	+	+
Плейлисты:			
создание/удаление	+	+	+
добавление/удаление треков	+	+	+
Рекомендации схожих произведений	+	+	+
Главная страница с рекомендациями	+	+	+

Продолжение таблицы 1

1	2	3	4
Поиск по:			
исполнителям	+	+	+
<i>дополнительная фильтрация*</i>	-	+	-
трекам	+	+	+
<i>дополнительная фильтрация*</i>	-	+	-
альбомам	+	+	+
<i>дополнительная фильтрация*</i>	-	+	-

+ — поддерживается

+ - — частично поддерживается

- — не поддерживается

* — дополнительная фильтрация результата поиска по тегам, продолжительности, дате выхода или др.

1.2.2 Особенности музыкальных сервисов

ВК Музыка. Музыкальная библиотека сервиса пополняется как пользователями, так и сотрудниками сервиса. Особенности сервиса:

- Большой объем библиотеки;
- Невозможность модерировать весь добавляемый пользователями контент;
- Слабая классификация песен по альбомам и исполнителям, поиск по такой библиотеке может привести к нерелевантным результатам.

SoundCloud. Библиотека пополняется пользователями сервиса, но каждый пользователь при этом регистрируется как автор контента (исполнитель). Особенности сервиса:

- Средний объем библиотеки, отсутствует контент исполнителей, которые не используют данный сервис;
- Хорошая классификация по исполнителям и альбомам.

Яндекс Музыка. Музыкальная библиотека сервиса пополняется сотрудниками сервиса. Особенности сервиса:

- Хорошая классификация по исполнителям и альбомам;
- Подробная информация о добавляемом контенте;
- Высокий уровень контроля качества контента.

1.3 Формирование бизнес-требований

Сервис должен хранить собственную библиотеку свободной музыки (copyright) и предоставлять пользователям возможность ее поиска и прослушивания, без необходимости платить за подписку для прослушивания музыки в высоком качестве.

Сервис должен предоставлять пользователям, являющимся администраторами, возможность пополнения и редактирования библиотеки музыки с использованием предоставленных функций API.

1.4 Формирование функциональных требований

Составлены функциональные требования к системе, они разделены на группы.

Пользователи. Система должна предоставлять возможность:

- Создания пользователей;
- Аутентификации и авторизации пользователей;
- Разделения пользователей на роли:
 - Обычный пользователь;
 - Администратор.
- Изменения и просмотра личной информации о себе и других пользователях, включая:
 - логин;
 - имя (опционально);
 - фамилию (опционально);

- адрес электронной почты (опционально);
- пользовательское описание (опционально);
- изображение пользователя (опционально);
- просмотр понравившихся пользователю произведений.

Треки. Система должна предоставлять возможность:

- Прослушивания треков с использованием стриминга, получения информации о треках, включая:

- название трека;
- продолжительность трека;
- альбом, которому принадлежит данный трек;
- исполнителей данного трека;
- обложку трека;
- жанровые теги.

- Добавления, удаления и редактирования треков. *

Альбомы. Система должна предоставлять возможность:

- Получения информации об альбоме, включая:

- название альбома;
- дату релиза;
- обложку альбома;
- исполнителей данного альбома;
- треки, входящие в данный альбом.

- Добавления, удаления и редактирования альбомов. *

Исполнители. Система должна предоставлять возможность:

- Получения информации об исполнителях, включая:

- псевдоним исполнителя;
- фото исполнителя;

- альбомы исполнителя;
- треки исполнителя.
- Добавления, удаления и редактирования исполнителей. *

Плейлисты. Система должна предоставлять возможность:

- Получения информации о плейлистах пользователя и содержащихся в них треках;
- Управления плейлистами пользователя, включающее**:
- добавление и удаление треков плейлиста;
- создание и удаление плейлистов;
- редактирование обложки плейлиста.

Очередь треков. Система должна предоставлять возможность:

- Сохранения очереди треков, даже при смене пользователем устройства;
- Управления очередью треков для каждого пользователя.

Система оценок («лайков»). Система должна предоставлять возможность:

- Установления оценки «нравится / не нравится» для:
- исполнителей,
- альбомов,
- треков.

Поиск. Система должна предоставлять возможность:

- Поиска по названию альбома:
- с фильтрацией по дате выхода.
- Поиска по имени исполнителя;
- Поиска по названиям треков:
- с фильтрацией по жанровым тегам.

Рекомендации к прослушиванию. Система должна предоставлять возможность:

- Получения «похожих» альбомов к определенному альбому;
- Получения случайных альбомов и исполнителей;
- Получения случайных песен определенного жанра;
- Получения последних выпущенных альбомов.

* — Только для администраторов

** — Только для плейлистов принадлежащих пользователю

1.5 Формирование модели прецедентов

Выделены три группы пользователей:

1. Неавторизованные пользователи, имеющие доступ к прослушиванию и поиску музыки;

2. Авторизованные пользователи, имеющие возможности аналогичные неавторизованным пользователям, а также возможность создавать плейлисты, управлять очередью треков, отмечать понравившиеся произведения, редактировать и просматривать свой профиль;

3. Администраторы, имеющие возможность пополнять и редактировать библиотеку произведений.

Диаграмма прецедентов приведена на рисунке 2.

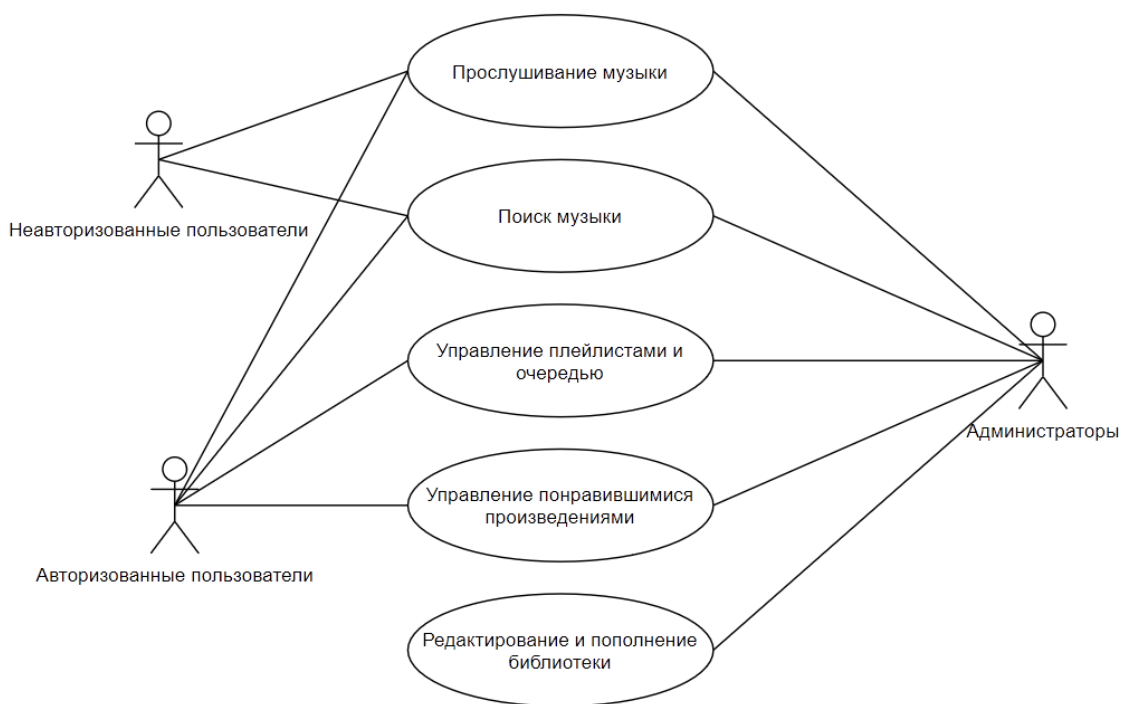


Рисунок 2 — Диаграмма прецедентов

1.6 Особенности архитектуры системы и моделирование предметной области

Разрабатываемое приложение имеет клиент-серверную архитектуру. «Клиент-сервер» — подход к разработке программного обеспечения, при котором «сервер» отвечает за выполнение запросов клиента, работу с базой данных и обработку всех сложных вычислений, а «клиент» является интерфейсом для выполнения этих запросов.

Серверное приложение должно предоставлять простое и эффективное API для работы с ним, в качестве подхода к его построению выбран архитектурный стиль REST [2]. REST - архитектурный стиль построения API, его главные особенности:

- Отсутствие состояния (“stateless”). Контекст взаимодействия сервера и клиента не сохраняется на сервере, любой запрос клиента включает в себя всю необходимую информацию для его выполнения. Такой подход позволяет повысить производительность приложения и упрощает его горизонтальное масштабирование;

- Уникальные идентификаторы ресурсов. Каждая единица данных имеет свой собственный URL, однозначно ее определяющий;
- Простые базовые операции над хранящимися объектами (CRUD):
 - CREATE – создание,
 - READ – получение,
 - UPDATE – обновление,
 - DELETE – удаление.
- «Самоописывающие» ответы. Любой ответ сервера содержит полную информацию о результате выполнения действия, либо возникшей ошибке;
- Управление ресурсами через представления. Форма хранения данных на сервере может не совпадать с представлением ресурса, получаемого клиентом (в форматах JSON, XML и т.д.).

2 ВЫБОР СРЕДСТВ РАЗРАБОТКИ ПРИЛОЖЕНИЯ

2.1 Язык программирования

Python – высокоуровневый интерпретируемый язык программирования общего назначения, ориентированный на повышение производительности и читаемости и качества кода [3]. Язык поддерживает объектно-ориентированное, структурное и императивное программирование.

Достоинствами языка являются:

- динамическая типизация,
- автоматическое управление памятью,
- простой, минималистичный синтаксис.

2.2 Фреймворк

Рассмотрены особенности фреймворков Flask [4], Django [5], FastAPI [6].

Flask. Микрофреймворк для создания веб-приложений, он предоставляет минимальное количество инструментов, имеет простую структуру и подходит для разработки небольших приложений.

Плюсы:

- Высокая производительность;
- Гибкая настройка дополнительных возможностей: ORM, авторизация, валидация.

Минусы:

- Нет поддержки асинхронности;
- Минимум функционала, предоставляемого по умолчанию, добавление необходимого сопровождается дополнительной конфигурацией каждого модуля.

Django. Высокоуровневый веб-фреймворк, использующий шаблон MVC.

Плюсы:

- Множество устанавливаемых модулей, дополняющих функционал основного фреймворка;

- Интегрированная система Django ORM.

Минусы:

- Средний уровень производительности;
- Избыточность возможностей фреймворка, многие встроенные возможности (встроенная авторизация, шаблонизатор) не будут использоваться в конечной системе и их невозможно удалить;

- Нет поддержки асинхронности.

FastAPI. Высокопроизводительный веб-фреймворк для построения API основанный на стандарте “Python type hints”.

Плюсы:

- Поддержка асинхронности с использованием интерфейса ASGI;
- Высокая производительность;
- Специализирован под разработку API;
- Поддержка моделей Pydantic для валидации ответов [7];
- Автоматическая генерация документации, предоставляемая в виде интерфейса SwaggerUI [8] по спецификации OpenAPI [9].

Минусы:

- Малый функционал, предоставляемый по умолчанию;
- Фреймворк достаточно новый, первый релиз состоялся в 2018 году, возможны недоработки и ошибки в работе.

В проекте выбран FastAPI по причине наличия в нем асинхронной обработки запросов и автоматической генерации документации OpenAPI.

2.3 СУБД

Документно-ориентированные. Это тип базы данных, предназначенный для хранения объектов в виде документов, десериализация которых возлагается на пользователя. Базы данных такого типа не используют связи между хранящимися данными, нарушая при этом согласованность данных в рамках NoSql подхода, что позволяет решить проблему масштабируемости базы данных и увеличить производительность.

В рамках разрабатываемой системы данный тип СУБД не является подходящим ввиду предполагаемой связности данных и необходимости в высоком уровне нормализации.

Объектно-реляционные. Среди данного типа СУБД наиболее известными являются PostgreSQL, MySql, SQLite.

PostgreSQL. PostgreSQL – свободная объектно-реляционная система управления базами данных поддерживаемая сообществом postgresql [9], написанная на языке Си. PostgreSQL в своих разработках используют компании Яндекс и Avito [10].

Плюсы:

- Полное соответствие требованиям ACID;
- Полная поддержка стандарта SQL ANSI;
- Поддержка дополнительных NoSql типов данных;
- Возможность дополнительной конфигурацией с использованием плагинов Postgresql.

Минусы:

- Сложность использования и конфигурации (в сравнении с рассматриваемыми СУБД).

MySQL. MySQL – свободная система управления реляционными базами данных, поддерживаемая корпорацией Oracle, написанная на языках Си и C++ [11].

Плюсы:

- Высокая скорость работы;
- Популярность.

Минусы:

- Неполное соответствие требованиям ACID;
- Неполная поддержка стандарта SQL ANSI.

SQLite. SQLite – встраиваемая СУБД хранящая все данные в одном файле, поставляется в виде библиотеки, которая встраивается в код приложения работающего с СУБД [12]. SQLite используется в следующих проектах [13]: Фреймворк QT, Mozilla Firefox, Skype, Viber.

Плюсы:

- Нет необходимости устанавливать дополнительный сервер СУБД, т.к. SQLite является встраиваемой системой;

Минусы:

- Нет возможности горизонтально масштабировать базу данных;
- Отсутствие пользователей для управления базой данных;
- Малое количество поддерживаемых типов данных;

СУБД SQLite не подходит для крупных приложений с большим количеством хранящихся данных. PostgreSQL была выбрана по причине наличия в ней поддержки NoSQL типов данных и наличия опыта работы с ней.

2.4 Поисковая система

Поисковые движки используются для полнотекстового поиска с учетом морфологии языка, предоставляя возможности фильтрации и ранжирования (оценки совпадения) результатов. Для реализации поиска и фильтрации рассмотрены поисковые движки: Elasticsearch [14], Postgresql [15], TypeSense [16]. Для сравнения использована статистика, собранная gigasearch.co [17]. Сравнение скорости индексирования приведено на рисунке 3, сравнение задержки в выполнении запросов приведено на рисунке 4, сравнительная характеристика приведена в таблице 2.

ElasticSearch. Программная поисковая система, основанная на библиотеке полнотекстового поиска Apache Lucene. Поддерживает многопоточность, горизонтально масштабируется. Предоставляет RestAPI для конфигурации и выполнения запросов.

Плюсы:

- Поддержка большого числа языков;
- Высокая скорость индексирования;
- Высокая скорость поиска;
- Гибкость в построении поисковых запросов.

Минусы:

- Необходимость дублировать данные, по которым будет происходить поиск в базу данных движка для построения индексов.

Postgresql. Система PostgreSQL имеет встроенные возможности для конфигурации полнотекстового поиска.

Плюсы:

- Нет необходимости в конфигурации дополнительного сервера, поиск можно реализовать средствами базы данных;

- Как следствие предыдущего пункта, нет необходимости дублировать данные;

- Высокая скорость индексирования;

- Высокая скорость поиска*.

- * Низкая скорость обработки запросов, содержащих одно слово

Минусы:

- Трудность настройки для сложных запросов с фильтрацией и поиску по нескольким полям;

- Ограниченная поддержка языков, отсутствует поддержка иероглифов.

TypeSense. Простой в использовании, «in-ram» поисковый движок. Особенностью TypeSense является хранение всех индексов в оперативной памяти, что обеспечивает низкую задержку выполнения поисковых запросов.

Плюсы:

- Высокая скорость индексации;

- Высокая скорость поиска.

Минусы:

- Размер хранилища ограничен объемом доступной RAM;

- Необходимость дублировать данные, по которым будет происходить поиск в базу данных движка для построения индексов.

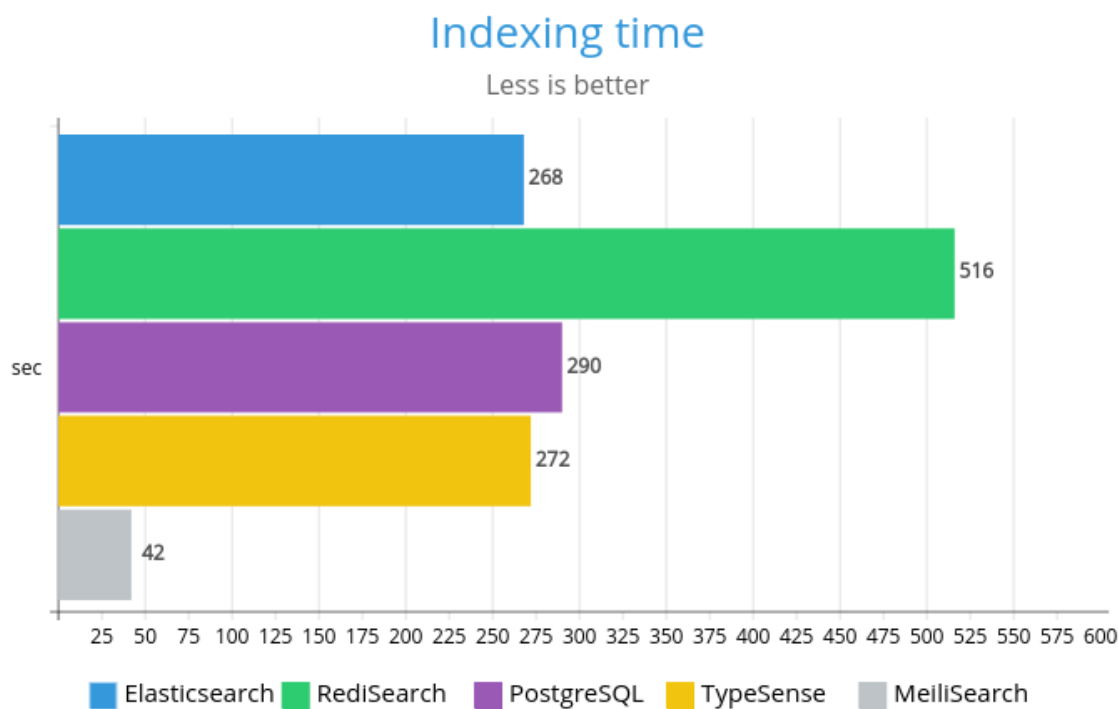


Рисунок 3 — Сравнение скорости индексирования поисковых систем

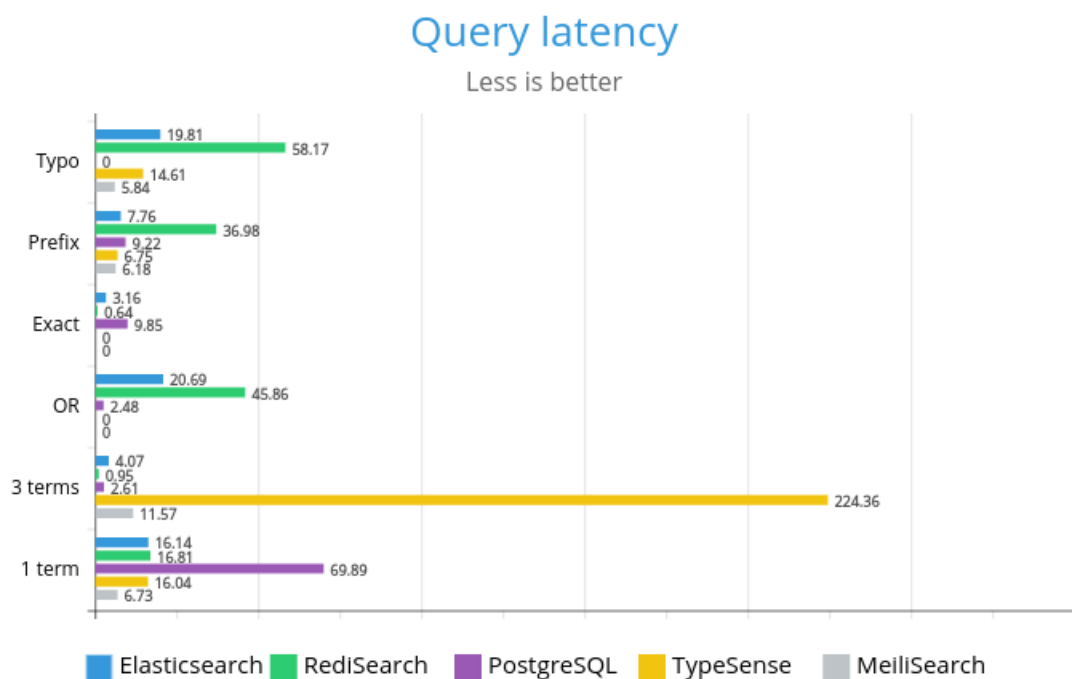


Рисунок 4 — Сравнение задержки выполнения запросов поисковых систем

Таблица 2 — Сравнительная характеристика поисковых систем

Тест	Elasticsearch	PostgreSQL	TypeSense
Индексирование			
- время (с)	268.000	290.000	272.000
- пропускная способность (документ/с)	23644.000	21827.000	23258.000
Запрос: 1 слово (мс)	16.140	69.890	16.040
Запрос: 3 слова (мс)	4.070	2.610	224.360
Запрос: 3 слова "ИЛИ" (мс)	20.690	2.480	-
Запрос: точная фраза (мс)	3.160	9.850	-
Запрос: 1 слово, автокомплит (мс)	7.760	9.220	6.750
Запрос с опечаткой (мс)	19.810	-	14.610

Так как поиск необходимо осуществлять по большому количеству треков, использование системы TypeSense отклоняется. Также необходима поддержка множества языков (названия треков, альбомов и исполнителей может быть любым). Для обеспечения поиска выбрана система Elasticsearch, ввиду гибкости и простоты ее настройки.

2.5 Веб-Сервер

Для запуска ASGI приложения необходим подходящий асинхронный сервер, среди таких: HyperCorn [18] и Uvicorn [19].

Различие между ними минимальны, оба сервера основаны на библиотеке trio [20] и работают с использованием цикла событий uvloop [21]. В работе используется сервер Uvicorn, т.к. он рекомендуется разработчиками FastAPI, как и запуск сервера в многопоточном режиме с использованием gunicorn [22].

2.6 Дополнительные средства

Sqllalchemy ORM. ORM (Object Relation Mapping) – технология, позволяющая работать с базами данных, используя парадигмы ООП [23]. Технология работает по принципу выстраивания «виртуальной» объектной базы данных, представляя таблицы, строки и колонки таблиц в виде объектов. ORM позволяет отойти от использования SQL запросов, предоставляя эквивалентные возможности конструирования запросов при помощи собственных объектов и методов.

Преимуществами использования SQLAlchemy перед другими ORM является:

- Гибкость в построении запросов;
- Простой способ описывания моделей;
- Поддержка большого числа баз данных с возможностью простого переключения между ними. Поддержка PostgreSQL, включая специфичные для нее типы данных и возможности.

Alembic DB Migration CLI. Alembic – инструмент для написания миграций баз данных, позволяющий удобно описывать и вносить изменения в текущую структуру базы данных без использования SQL напрямую [24], особенностями данного инструмента являются:

- Простота использования;
- Возможность написания миграций вручную или при помощи автоматической генерации на основе предоставляемого файла моделей SQLAlchemy;
- Поддержка и разработка инструмента осуществляется автором SQLAlchemy ORM.

VSC Git. VSC (Version Control System) – система контроля версий, это программное обеспечение для работы с изменяющимися данными (исходным кодом, файлами конфигураций и пр.) позволяющее хранить версии данных и при необходимости возвращаться к определенной из них.

Git – распределенная система контроля версий, обеспечивающая высокую производительность при работе с версиями, созданием веток и их слиянием, имеет удобный и простой интерфейс.

Средство тестирования pytest. Фреймворк для написания, выполнения и поддержки тестов [25], обладает следующими особенностями:

- Простое создание тестов с применением встроенной в python функции `assert`;
- Автоматическое указание места возникновения ошибки с выводом кода;
- Модульные «фикстуры» для управления небольшими тестами;
- Более 800 дополнительных плагинов для настройки.

Docker. Docker – ПО для автоматизации создания и развертки контейнеров в системе Linux [26]. Позволяет упаковать приложение в независимый контейнер, работающий в изолированной среде и имеющей доступ только к собственной файловой системе. Подобный подход дает возможность легко переносить настроенные приложения со множеством зависимостей на любую машину, поддерживающую работу docker. Т.к. система не использует виртуальные машины, а изолирует запускаемые процессы, то производительность с использованием docker контейнеров намного выше.

3 АРХИТЕКТУРНОЕ ПРОЕКТИРОВАНИЕ

3.1 Общая архитектура и взаимодействие компонентов

Архитектура системы изображена на рисунке 5.

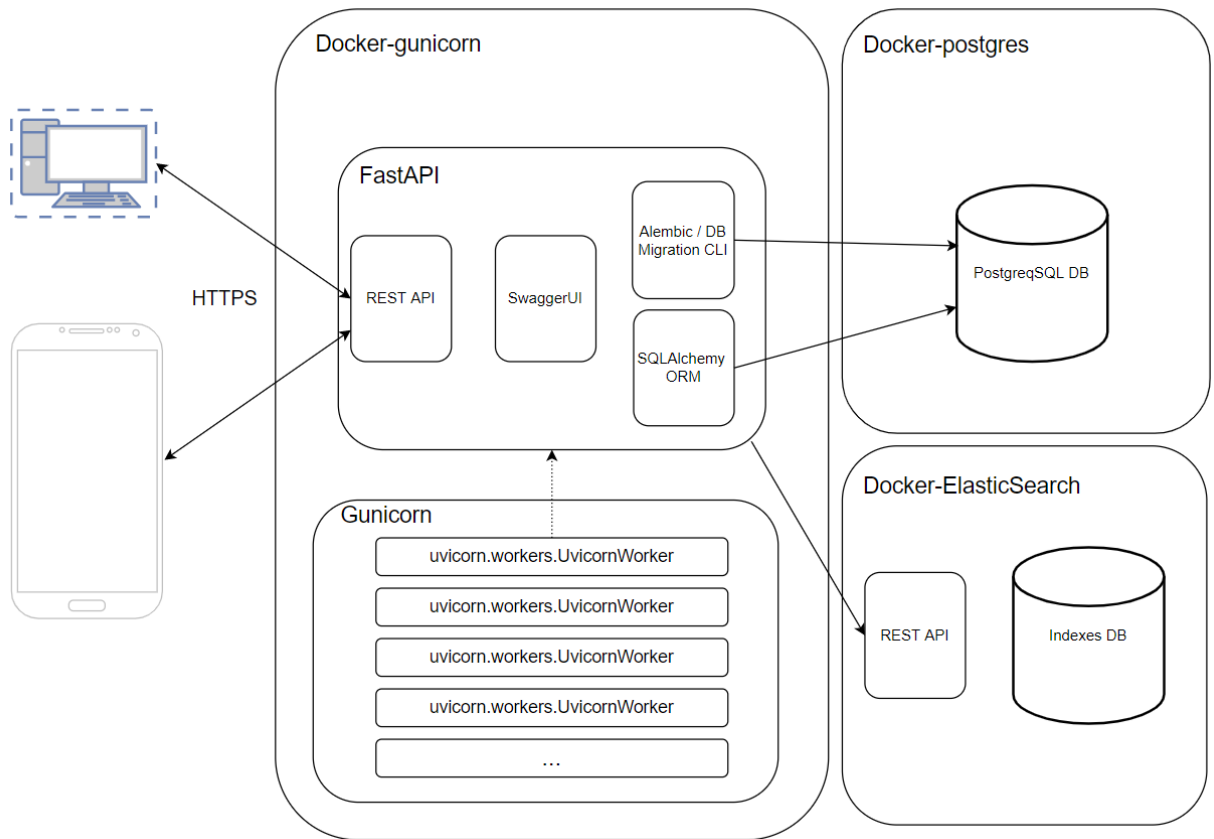


Рисунок 5 — Взаимодействие компонентов системы

Система состоит из трех независимых docker-контейнеров:

1. Docker-gunicorn – содержит веб-сервер gunicorn. Gunicorn запускает в нескольких потоках uvicorn в качестве server-worker, каждый из которых запускает приложение FastAPI, которое отвечает за раздачу статического контента (изображения пользователей, альбомов, исполнителей и т.д.) и стриминг треков, оно предоставляет:

а) REST API для взаимодействия с пользователями;

б) SwaggerUI для генерации автоматической документации к REST API;

2. Docker-postgres – содержит базу данных PostgreSQL;

3. Docker-ElasticSearch – содержит сервер и базу данных поискового движка ElasticSearch, который предоставляет свой RestAPI для работы.

3.2 Структуры данных

3.2.1 Объектная модель

В результате анализа предметной области построена объектная модель системы, рисунок 6.

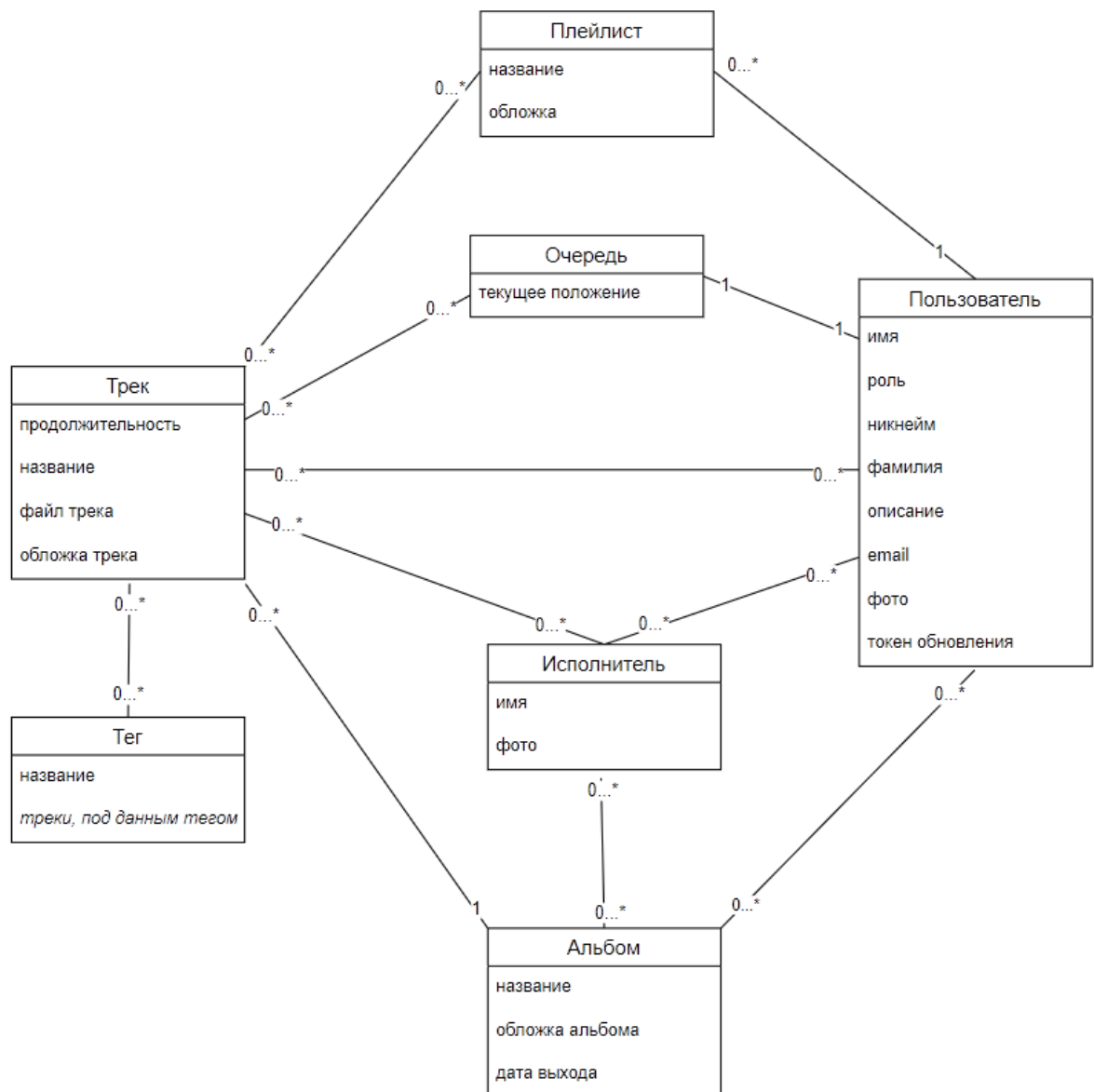


Рисунок 6 — Объектная модель системы

Объектные модели отдельных сущностей системы приведены ниже.

Пользователь

Объектная модель сущности «Пользователь» приведена на рисунке 7.

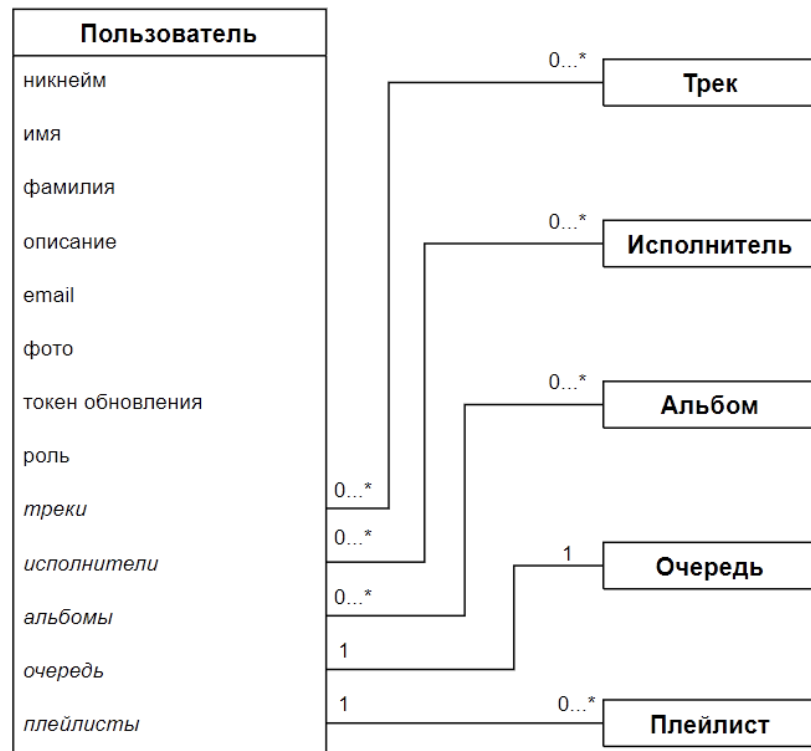


Рисунок 7 — Объектная модель сущности «Пользователь»

Описание полей сущности «Пользователь»:

- логин — псевдоним, выбранный пользователем для авторизации и отображения в профиле;
- имя — имя пользователя, для отображения в профиле;
- фамилия — фамилия пользователя, для отображения в профиле;
- описание — пользовательское описание «о себе», для отображения в профиле;
- email — адрес электронной почты пользователя;
- фото — изображение пользователя;

- токен обновления – токен обновления, использующийся в механизме аутентификации;
- роль – метка, обозначающая является ли данный пользователь администратором;
- треки – треки, оцененные пользователем как понравившиеся;
- исполнители – исполнители, оцененные пользователем как понравившиеся;
- альбомы – альбомы, оцененные пользователем как понравившиеся;
- очередь – текущая очередь треков и позиция прослушиваемого трека;
- плейлисты – созданные пользователем плейлисты с треками.

Плейлист

Объектная модель сущности «Плейлист» приведена на рисунке 8.

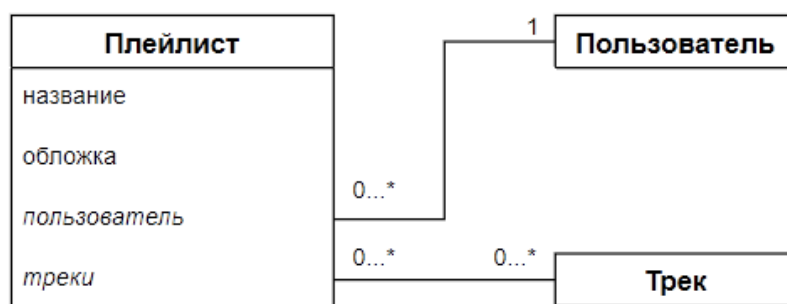


Рисунок 8 — Объектная модель сущности «Плейлист»

Описание полей сущности «Плейлист»:

- название – имя плейлиста, указанное пользователем;
- обложка – изображение, указанное пользователем для плейлиста;
- пользователь – создатель данного плейлиста;
- треки – треки, содержащиеся в данном плейлисте.

Очередь

Объектная модель сущности «Очередь» приведена на рисунке 9.

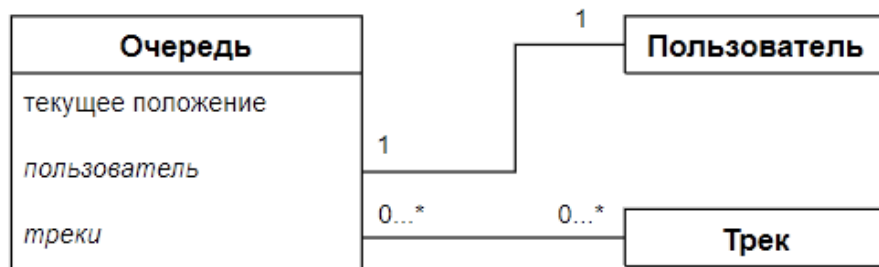


Рисунок 9 — Объектная модель сущности «Очередь»

Описание полей сущности «Очередь»:

- текущее положение – номер текущего трека в очереди;
- пользователь – пользователь, к которому относится данная очередь;
- треки – треки, содержащиеся в данной очереди.

Исполнитель

Объектная модель сущности «Исполнитель» приведена на рисунке 10.

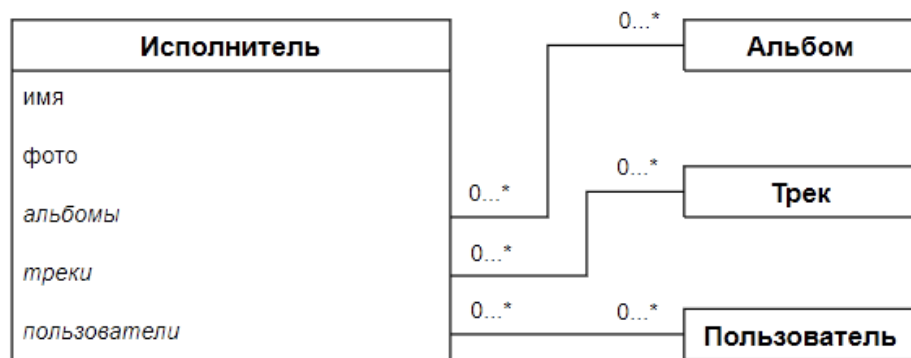


Рисунок 10 — Объектная модель сущности «Исполнитель»

Описание полей сущности «Исполнитель»:

- имя – псевдоним или имя исполнителя;

- фото – фото исполнителя;
- альбомы – альбомы, принадлежащие данному исполнителю;
- треки – треки, принадлежащие данному исполнителю;
- пользователи – пользователи, оценившие данного исполнителя как понравившегося.

Трек

Объектная модель сущности «Трек» приведена на рисунке 11.

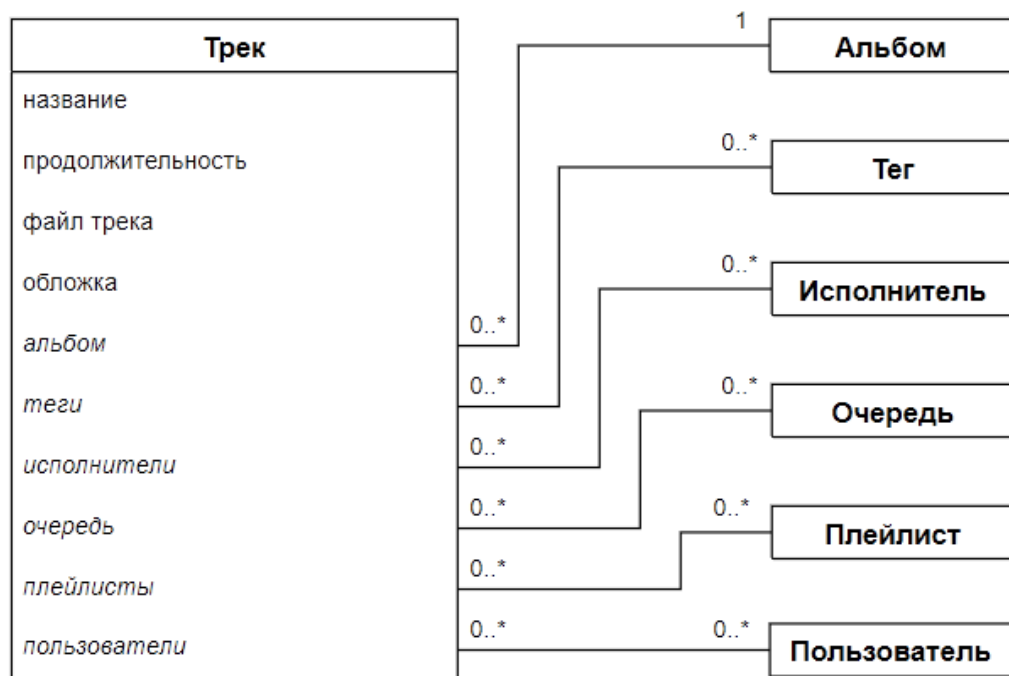


Рисунок 11 — Объектная модель сущности «Трек»

Описание полей сущности «Трек»:

- название – название трека;
- продолжительность – длительность (файла) трека;
- обложка – изображение обложки трека;
- альбом – альбом, которому принадлежит данный трек;
- теги – жанровые теги к которым относится данный трек;

- исполнители – исполнители данного трека;
- очередь – очередь в которой содержится данный трек;
- плейлисты – плейлисты в которых содержится данный трек;
- пользователи – пользователи, оценившие данный трек как понравившийся.

Альбом

Объектная модель сущности «Альбом» приведена на рисунке 12.

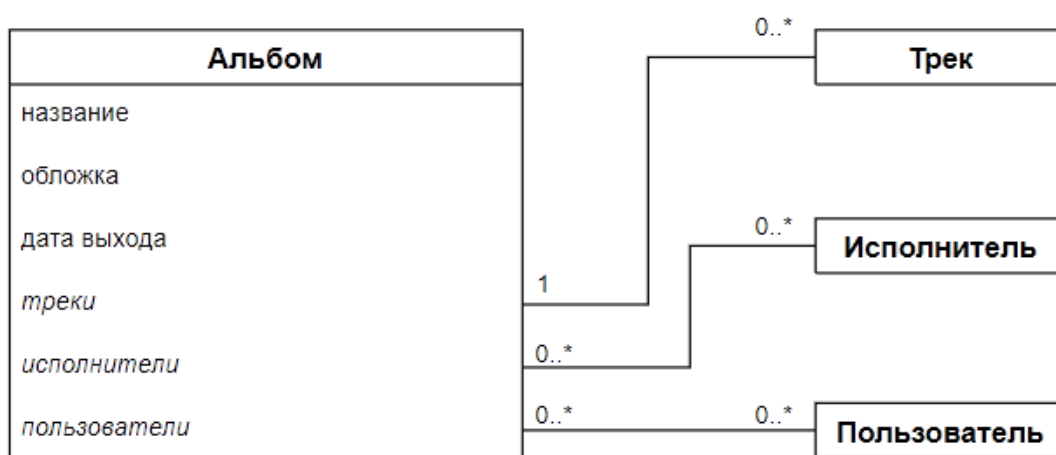


Рисунок 12 — Объектная модель сущности «Альбом»

Описание полей сущности «Альбом»:

- название – название данного альбома;
- обложка – изображение обложки трека;
- дата выхода – дата релиза альбома;
- треки – треки, содержащиеся в данном альбоме;
- исполнители – исполнители данного альбома;
- пользователи – пользователи, оценившие данный альбом как понравившийся.

Тег

Объектная модель сущности «Тег» приведена на рисунке 13.

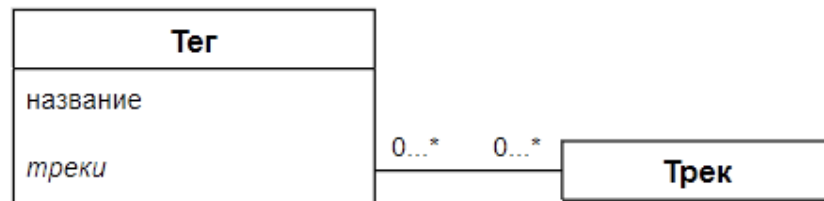


Рисунок 13 — Объектная модель сущности «Тег»

Описание полей сущности «Тег»:

- название – название жанра, который обозначает данный трек;
- треки - треки, опубликованные под данным тегом.

3.2.2 Структура базы данных

На основе сформированных объектных моделей спроектирована база данных, структура которой представлена на рисунке 14.

Объектные модели в базе данных представлены в качестве таблиц, атрибуты моделей – как колонки этих таблиц определенных типов.

Отношения один к одному, один ко многим, многие к одному представлены в виде внешних ключей в таблицах (отношение альбомов к трекам, отношение очереди к пользователю, отношение плейлистов к пользователю).

Отношения многие ко многим представлены с использованием промежуточных таблиц (song_artist, album_artist, user_artist, user_album, user_song, song_tag).

Таблица album_relations служит для хранения информации о схожести альбомов друг с другом и используется в рекомендации пользователю схожих альбомов, принцип ее заполнения описан в главе 3.

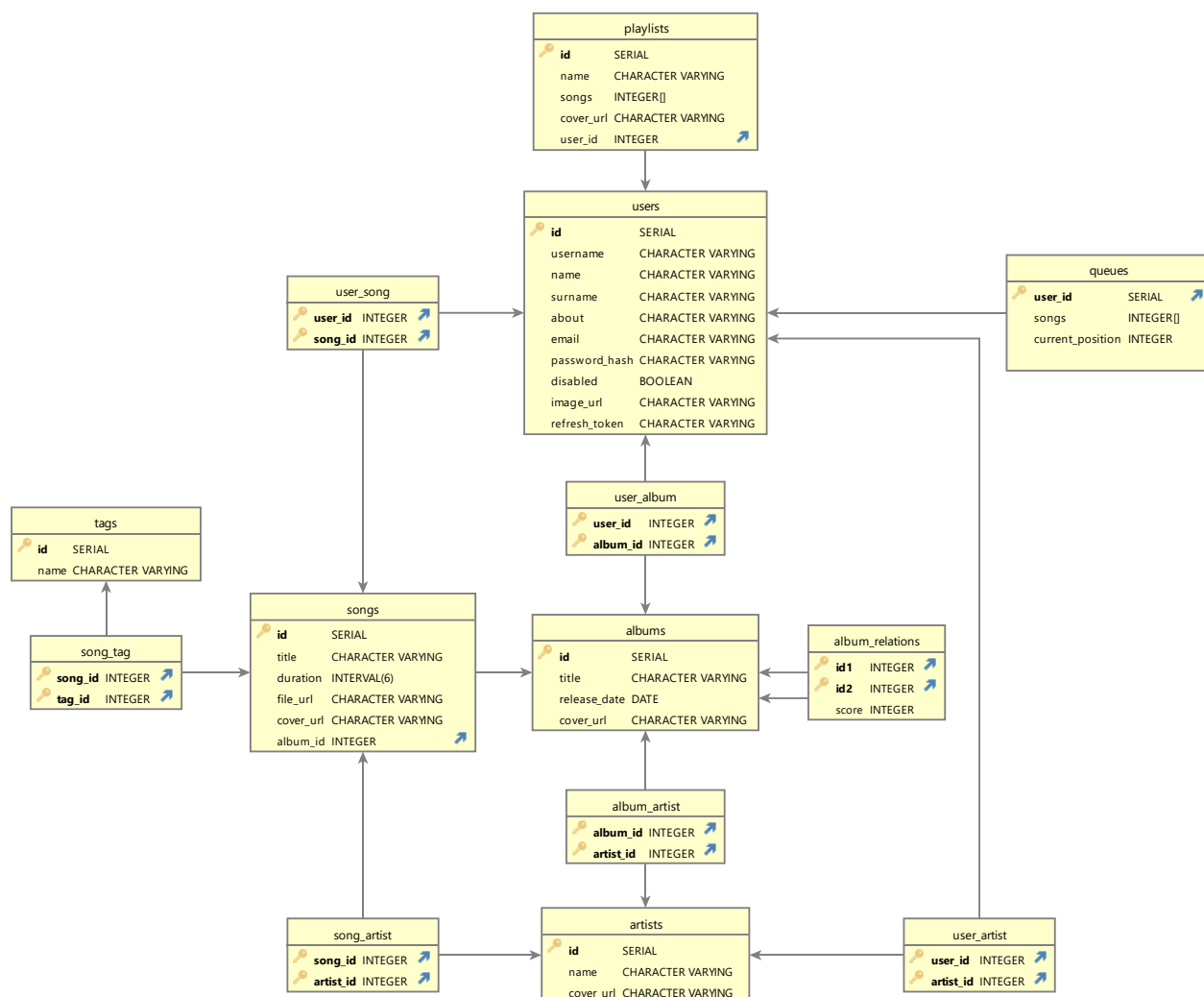


Рисунок 14 — ERD диаграмма базы данных

Для хранения списка песен в плейлистах и очередях используется массив id песен. Для ускорения запросов используются индексы для всех внешних и основных ключей.

3.3 Алгоритмы

3.3.1 Аутентификация

Для соблюдения правил RESTful подхода идентификация, аутентификация и авторизация пользователей реализована с использованием токенов (Token-Based Authentication), при таком подходе для получения доступа к ресурсам пользователь передает не логин и пароль, а выданный ему токен доступа (access_token).

В качестве токена доступа используется JWT токен, его стандарт описан в документе RFC 7519 [27]. JWT токен это JSON объект состоящий из трех частей:

- заголовка (header);
- полезной нагрузки (payload);
- подписи (signature).

В заголовке указывается алгоритм хеширования для создания подписи и тип токена «jwt», в полезной нагрузке – информация необходимая для идентификации и авторизации пользователя (id, логин, роль), а также время жизни токена (поле exp, время жизни access токена низкое, 30 минут), подпись – хеш генерируемый основываясь на секретном ключе и header с payload.

Все части токена кодируются при помощи алгоритма base64url и соединяются через точку.

Пример JWT токена приведен на рисунке 15.



Рисунок 15 — JWT токен, кодированный и декодированный

Последовательность действий для получения токена доступа изображена на рисунке 16.

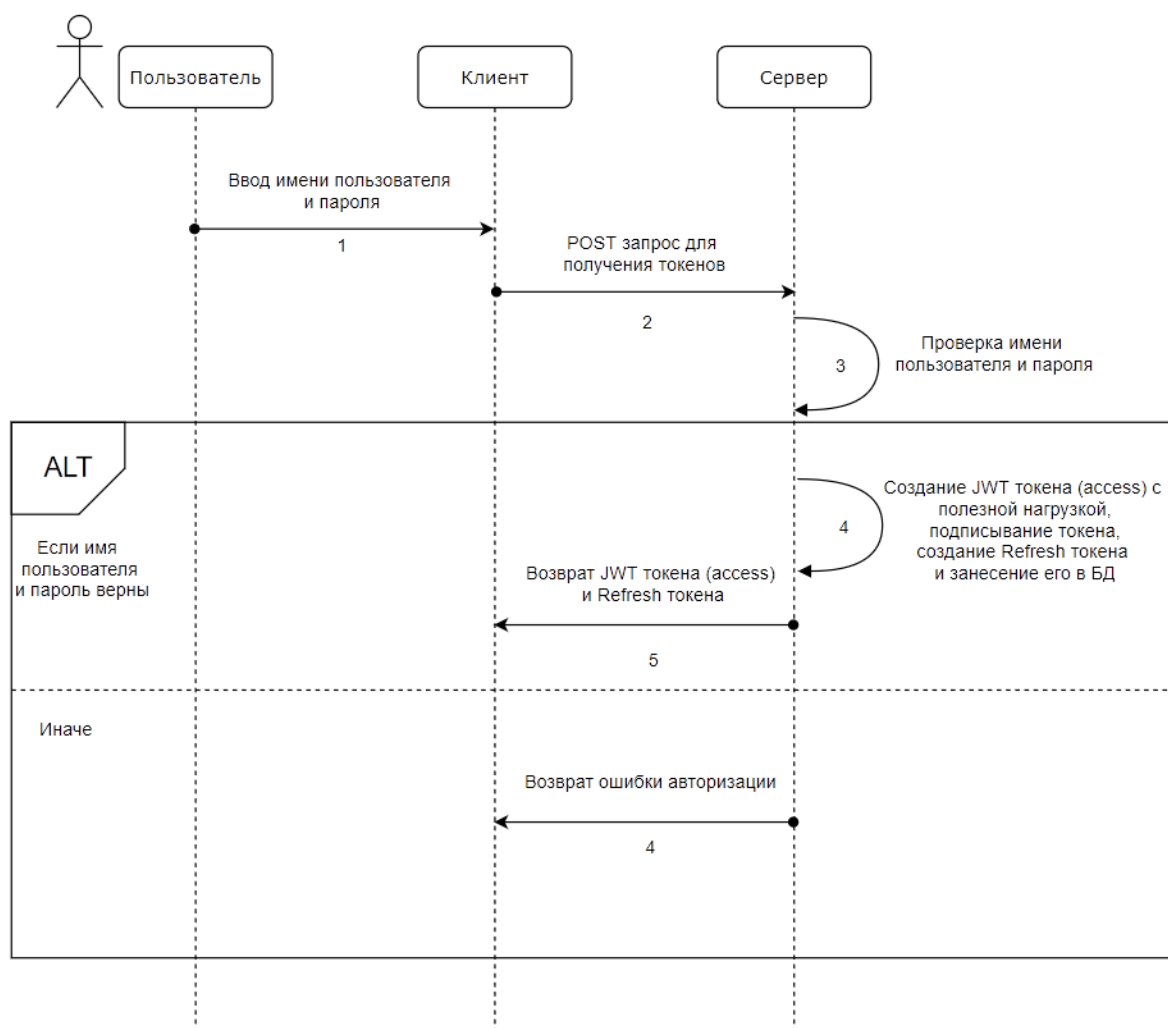


Рисунок 16 — Диаграмма последовательности, «Получение токенов»

Кроме токена доступа пользователь также получает токен обновления (действие 4), токен обновления — это строка, сгенерированная алгоритмом uuid4 и хранящаяся в базе данных для каждого пользователя. Токен обновления используется для получения нового токена доступа, в случае истечения его времени жизни.

Использование токена обновления позволяет не вводить логин и пароль (или хранить их на стороне клиента) при каждом истечении времени жизни токена доступа, а получать новый токен доступа, используя одноразовый токен обновления с большим временем жизни (от года и выше).

При необходимости обновления токена доступа клиент обращается к специальной конечной точке API, указывая в запросе свой токен обновления.

При наличии токена обновления в базе данных пользователь получает новый токен доступа и новый токен обновления, который также обновляется в базе данных.

Последовательность действий при использовании токенов для получения доступа к защищенным ресурсам изображена на рисунке 17.

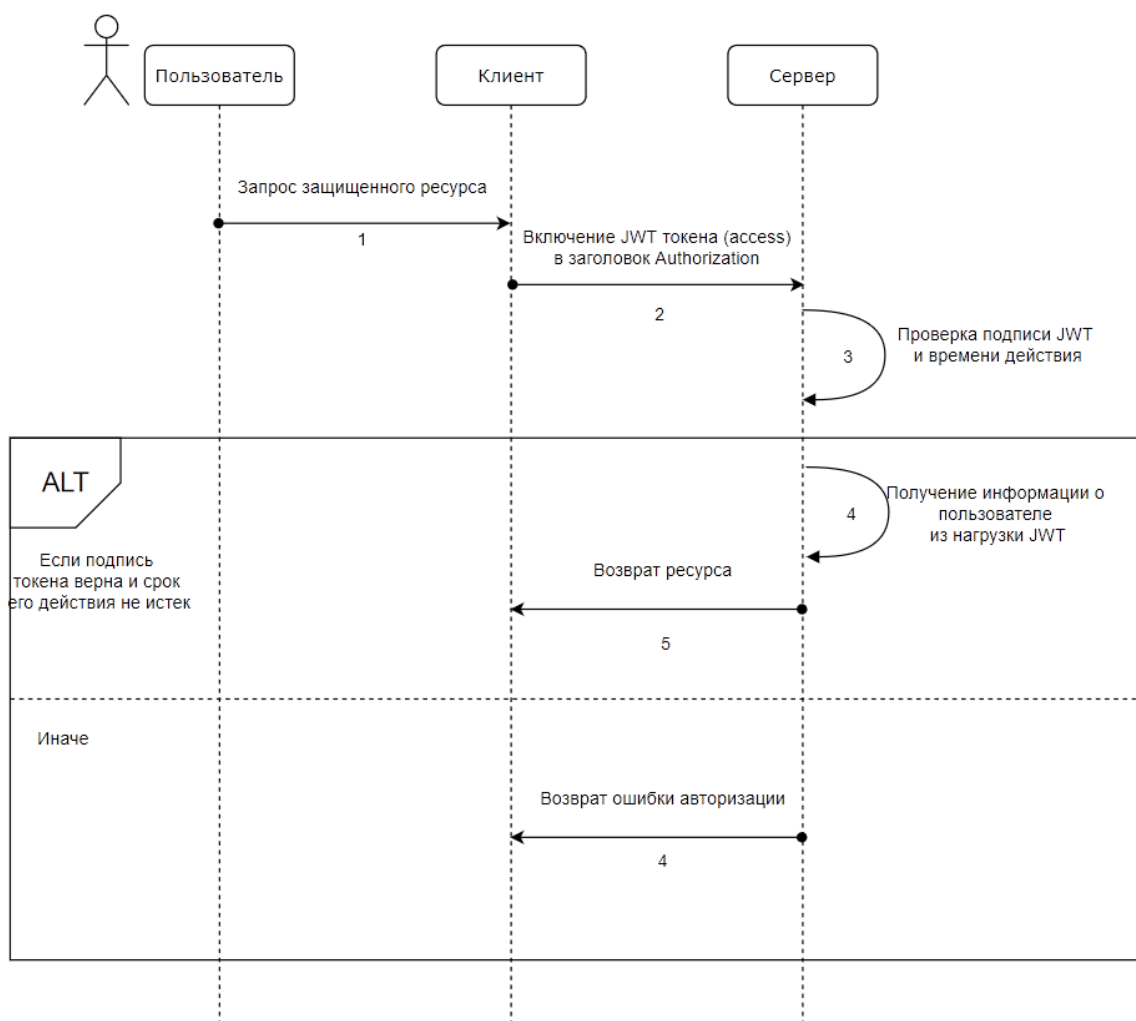


Рисунок 17 — Диаграмма последовательности, «Использование токенов»

Использование такого подхода позволяет уменьшить количество запросов к базе данных для проверки реквизитов пользователя и упростить аутентификацию на стороне клиентских приложений.

3.3.2 Стриминг контента

Для возможности стриминга файлов треков реализована поддержка частичного Get-запроса на стороне сервера. Частичные Get-запросы позволяют скачивать лишь указанную часть ресурса (работают с любыми типами файлов). Частичные Get-запросы отличаются от обычных наличием заголовка Range с указанным байтовым диапазоном, который необходимо получить. Сервер, поддерживающий данный тип запросов, отвечает на них с использованием статус кода HTTP 206 Partial Content и передаваемыми байтами запрашиваемого файла в теле ответа. Данный тип запросов описан в документе RFC 2616, в секции 14.35 [28].

По умолчанию поддержка запросов данного типа в использованной FastAPI библиотеки Starlette для раздачи статического контента отсутствовала, сервер игнорировал заголовки типа Range и возвращал файлы целиком со статус кодом 200, это продемонстрировано на рисунке 18.

В таком случае перемотка трека невозможна до его полной загрузки. На сервере для раздачи определенных типов файлов написан алгоритм обработки запросов данного типа, при получении запроса анализируется заголовок Range и определяется диапазон байт для передачи в теле ответа и указанием необходимых заголовков ответа, рисунок 19. Подробная реализация рассмотрена в главе 4.

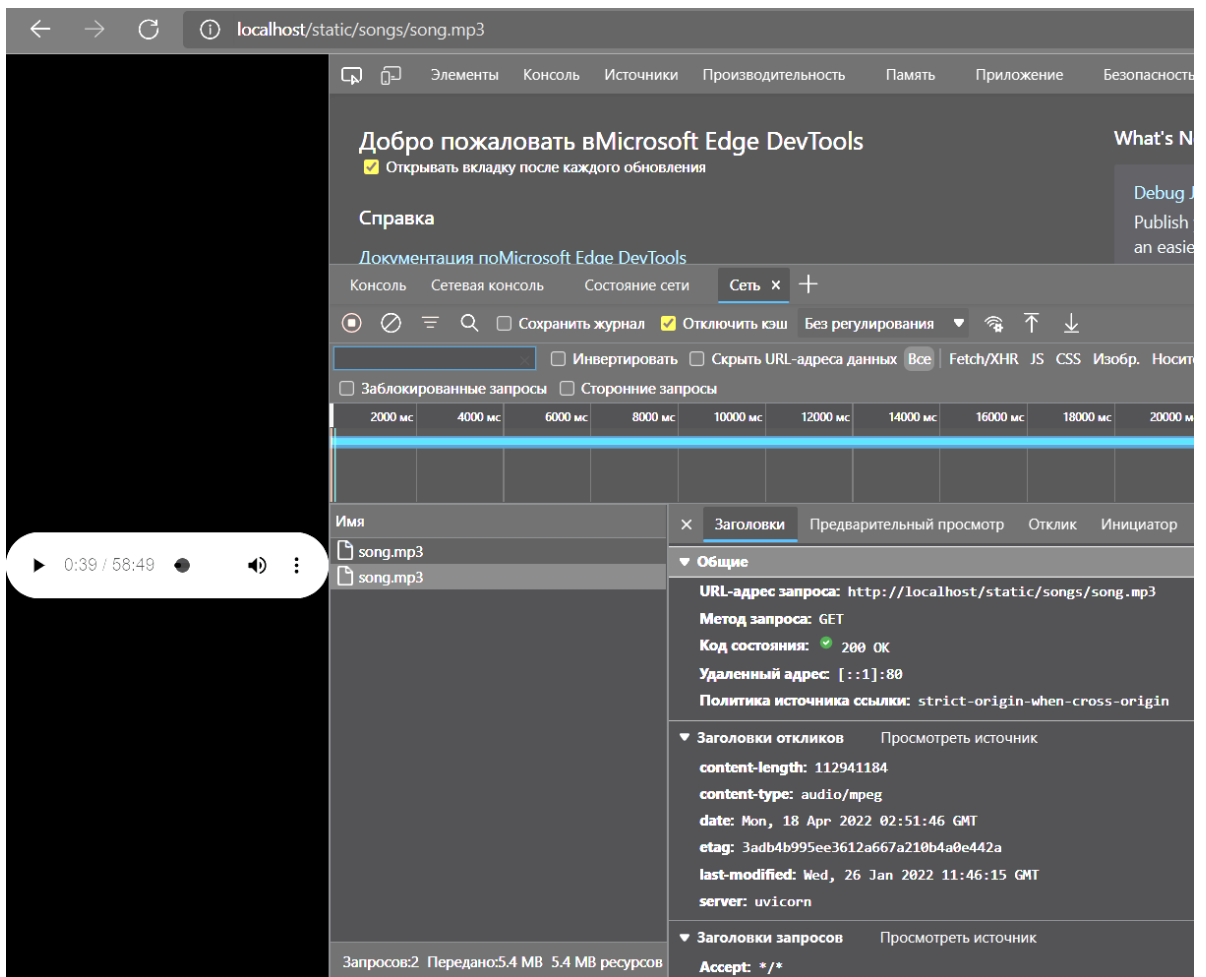


Рисунок 18 — Статическая раздача файла трека до реализации частичных запросов

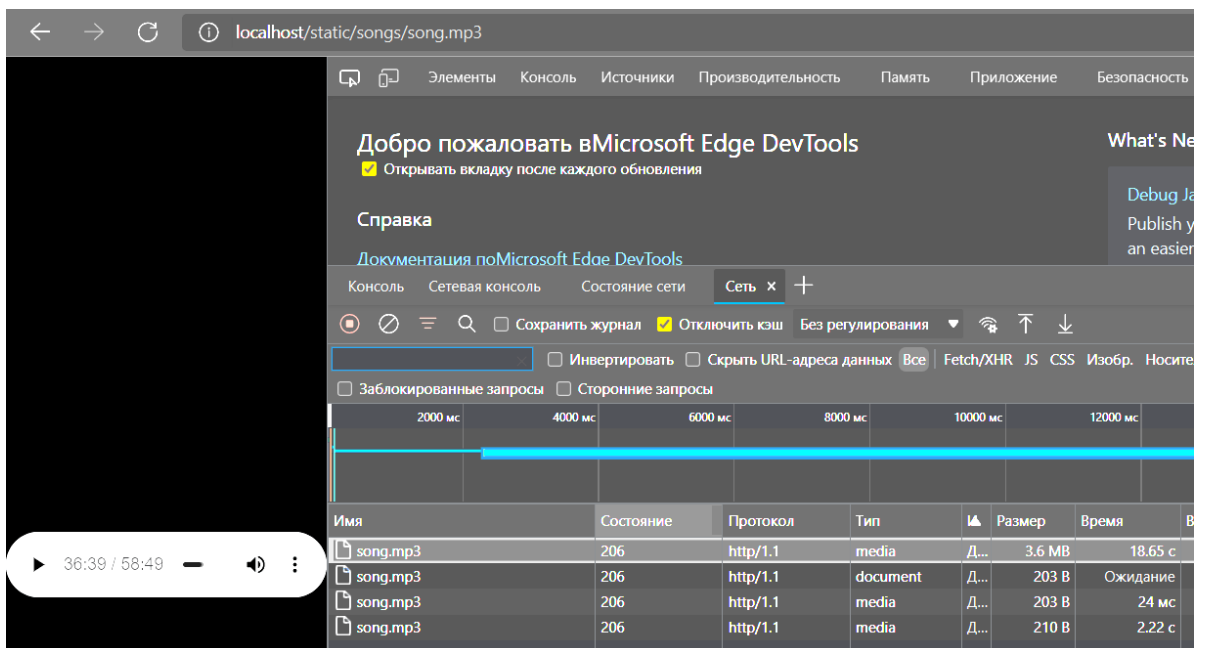


Рисунок 19 — Стриминг файла трека после реализации частичных запросов

После реализации частичных запросов файл трека отдается несколькими ответами типа HTTP 206, возможна его перемотка.

Подобная реализация стриминга может быть легко использована в веб-браузерах (поддержка реализована в большинстве браузеров [29]), так и просто реализуема для других клиентских приложений.

3.3.3 Получение рекомендаций на основе пользовательской статистики

Для получения рекомендаций схожих альбомов необходимо собрать статистику, и на основе понравившихся другим пользователям альбомов составить матрицу отношений между альбомами. Подобная матрица представлена на рисунке 20.

	Альбом №1	Альбом №2	Альбом №3	Альбом №4	
Альбом №1	-	4	5	2	
Альбом №2	4	-	0	3	■ ■ ■
Альбом №3	5	0	-	7	
Альбом №4	2	3	7	-	■ ■ ■

Рисунок 20 — Пример полной матрицы отношений альбомов

Каждый столбец и строка матрицы обозначают соответствующий номер альбома, на их пересечении – значение частоты, обозначающее как часто данные альбомы встречаются вместе в понравившихся пользователям альбомах.

Пример, на пересечении строки «Альбом №4» и столбца «Альбом №1» лежит число 2, значит данные альбомы вместе встречаются дважды среди различных альбомов, понравившихся пользователям.

Матрица является симметричной, следовательно, нет необходимости хранить симметричные значения, а также значения на диагонали, т.к. они не имеют смысла, как и хранение значений равных нулю. Сокращенная таблица представлена на рисунке 21.

	Альбом №1	Альбом №2	Альбом №3	Альбом №4	
Альбом №1					
Альбом №2	4				■ ■ ■
Альбом №3	5	0			
Альбом №4	2	3	7		■ ■ ■

Рисунок 21 — Пример сокращенной матрицы отношений альбомов

Для рекомендации альбома с номером N по такой матрице необходимо сравнить все значения, содержащиеся в столбцах, пересекающихся со строкой альбома N и найти наибольшие значения. Столбцы, соответствующие этим значениям – рекомендуемые альбомы, чем выше значение, тем точнее рекомендация.

Для хранения информации о рекомендациях используется таблица `album_relations`, содержащая поля:

- `id1`,

- id2,
- score.

где, id1 и id2 ссылаются на треки в таблице songs, score – значение частоты встречаемости.

Для заполнения таблицы необходимо для каждого пользователя (или некоторого количества) перебрать неповторяющиеся пары среди понравившихся ему альбомов и каждую пару id альбомов добавить в таблицу со значением score равным 1, или добавить единицу к значению score уже существующей записи. Для проверки работоспособности используются случайные пользователи со случайными понравившимися им альбомами.

3.4 API

В работе используется стороннее API Elasticsearch, для создания и выполнения запросов к нему используется низкоуровневый клиент в виде модуля для python “elasticsearch-py” [30]. Запросы к API приложения преобразуются в запросы к API поискового сервера в зависимости от их типов, шаблоны таких запросов описаны в главе 4.

Для возможности поиска в базу данных Elasticsearch дублируются данные из основной системы (только информация по которой осуществляется поиск), id продублированных объектов одинаковы в базе данных Elasticsearch и PostgreSQL. Elasticsearch в своих ответах возвращает только id найденных объектов, полная информация по которым получается из базы данных Postgres. При добавлении, удалении или обновлении объектов, информация обновляется и в поисковом движке тоже.

API Elasticsearch разбито на подгруппы, описание использованных в работе функций API приведено ниже.

Index API. Используется для управления отдельными индексами, настройками их отображений, псевдонимов и шаблонов. Индекс – оптимизированная форма хранения документов в системе Elasticsearch.

Используемые функции:

- Create index – создание индекса с указанным именем и отображением;
- Get index – получение информации о созданном индексе;
- Delete index – удаление указанного индекса.

Document API. Используется для управления документами, их созданием, удалением, обновлением и множественными операциями. Документы – объекты, хранящиеся в базе данных Elasticsearch.

Используемые функции:

- Index – добавление документа к индексу, или его обновление;
- Get – получение документа из индекса;
- Delete – удаление документа из индекса;
- Bulk – выполнение нескольких операций Document API одним вызовом.

Search API. Используется для поиска и агрегирования данных, хранящихся в индексах Elasticsearch. В работе используется только функция search, которая выполняет поиск в указанном индексе, используя запрос, написанный на языке Query DSL [31].

4 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ, СТРУКТУРА ПРОГРАММНОГО КОДА

4.1 Маршрутизация, конечные точки

Конечные точки API сгруппированы в отдельные роутеры, включающие в себя CRUD операции для каждого типа объектов, рисунок 22.

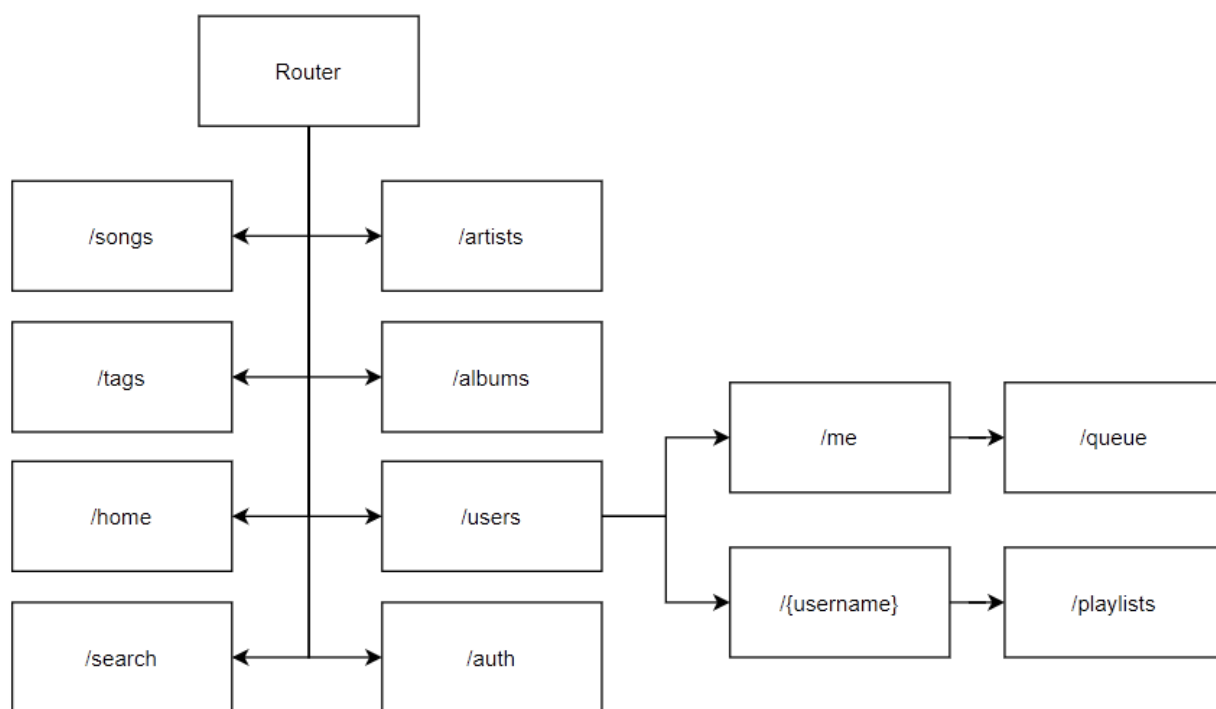


Рисунок 22 — Карта API

Полное описание всех конечных точек, их доступности, функционала и передаваемых значений приведено в приложении А.

4.2 Токены

Программная реализация механизма получения токенов доступа и обновления приведена на рисунке 23.

```

async def login_for_access_token( 1
    form_data: PasswordRequestForm = Depends(),
    db: Session = Depends(dependencies.get_db)
):
    user = crud_user.authenticate_user(form_data.username, form_data.password, db) 2
    if not user:
        raise HTTPException(
            status_code=401,
            detail="Incorrect username or password"
        )
    refresh_token = security.create_refresh_token() 3
    user.refresh_token = refresh_token
    db.commit()
    db.flush()
    data = {"sub": user.username, "type": "access", "id": user.id}
    if user.isAdmin:
        data["role"] = "admin"
    access_token = security.create_access_token(data=data) 4
    return {"access_token": access_token, "refresh_token": refresh_token, "token_type": "bearer"}

```

Рисунок 23 — Программная реализация генерации токенов доступа

Описание последовательности действий:

1. Указание URN конечной точки и Pydantic модели для валидации;
2. Проверка полученных реквизитов;
3. Создание токена обновления;
4. Генерация токена доступа, включение в полезную нагрузку токена имени пользователя, id пользователя, типа токена и роли пользователя.

Программная реализация механизма обновления токенов доступа и обновления приведена на рисунке 24. Описание последовательности действий:

1. Проверка переданного токена обновления на наличие в базе данных;
2. Создание нового токена обновления;
3. Создание нового токена доступа.

```

async def refresh_tokens(
    refresh_token: str,
    db: Session = Depends(dependencies.get_db)
):
    user = db.query(models.User).filter(models.User.refresh_token == refresh_token).first() 1
    if not user:
        raise HTTPException(
            status_code=401,
            detail="Invalid refresh token"
        )
    refresh_token_new = security.create_refresh_token() 2
    user.refresh_token = refresh_token_new
    db.commit()
    db.flush()
    data = {"sub": user.username, "type": "access", "id": user.id}
    if user.isAdmin:
        data["role"] = "admin"
    access_token = security.create_access_token(data=data) 3
    return {"access_token": access_token, "refresh_token": refresh_token_new, "token_type": "bearer"}

```

Рисунок 24 — Программная реализация генерации токенов доступа

4.3 CRUD

CRUD (Create, Read, Update, Delete) – стандартный набор операций используемых для работы с базами данных, все операции производимые с базой данных вынесены в отдельные crud функции и рассмотрены ниже.

4.3.1 Альбомы, исполнители, треки

CRUD классы альбомов, исполнителей и треков (AlbumCRUD, ArtistsCRUD, SongCRUD) реализуют интерфейс ItemBase, т.к. действия над ними схожи. Интерфейс включает в себя следующие методы:

- get – получение одного объекта по id;
- create – создание нового объекта;
- update – обновление объекта по id;
- delete – удаление объекта по id;
- get_all – получение объектов количества limit подряд, с пропуском skip объектов;
- get_list – получение списка объектов по переданному списку id;

- `get_random` – получение случайного объектов количества `limit`;
- `get_liked` – получение объектов понравившихся пользователю `user`;
- `like` – поставить «лайк» или «дизлайк» объекту по `id`, для указанного пользователя пользователя.

Помимо указанных методов у класса `AlbumCRUD` имеется метод `get_recommendation`, возвращающий рекомендации к указанному `id` альбома, рисунок 25.

```

async def get_recommendation(
    self,
    db: Session,
    id,
    skip: int = 0,
    limit: int = 5,
    current_user: Optional[schemas.User] = None
):
    results = db.query(models.AlbumRelations).\ 1
        filter(or_(models.AlbumRelations.id1==id,models.AlbumRelations.id2==id)).\ 2
        order_by(models.AlbumRelations.score.desc()).\ 3
        offset(skip).\ 4
        limit(limit).\
        all()
    albums_id = []
    for result in results: 5
        if result.id1 == id:
            albums_id.append(result.id2)
        else:
            albums_id.append(result.id1)
    return await self.get_list(db=db, id_list=albums_id, current_user=current_user)

```

Рисунок 25 — Метод получения рекомендаций к альбому (схожих альбомов)

Описание последовательности действий:

1. Создание запроса к объектам таблицы отношений альбомов при помощи ORM;
2. Фильтрация по совпадению поля `id1` или `id2` с указанным `id` альбома;
3. Сортировка результатов по убыванию значения частоты повторений (`score`);
4. Указание количества пропускаемых значений (`skip`) и ограничения (`limit`) получаемых объектов;

5. Составление списка из id полученных альбомов.

4.3.2 Очереди

Очередь треков реализована на массиве, с сохранением текущей позиции.

Класс очереди предоставляет стандартные операции над очередью.

Класс QueueCRUD содержит следующие методы:

- `get(user, ...)` – получение очереди `user`;
- `add(user, id, ...)` – добавление трека по `id` в конец очереди `user`;
- `add_list(user, song_list, ...)` – добавление списка треков `song_list` в конец очереди `user`;
- `delete(user, position, ...)` – удаление трека из очереди `user` находящегося на позиции `position`;
- `current(user, ...)` – получение текущего трека очереди `user`;
- `next(user, ...)` – получение следующего трека очереди `user`, перемещение позиции в очереди;
- `prev(user, ...)` – получение следующего трека очереди `user`, перемещение позиции в очереди;
- `replace(user, song_list, ...)` – замена очереди `user` списком треков `song_list`, обнуление позиции в очереди;
- `clear(user, ...)` – очистка очереди указанного пользователя;
- `set(user, position ...)` – установка позиции очереди `user` на `position`.

Код метода получения текущего трека приведен на рисунке 26.

```

async def current(self, db, user: schemas.User):
    db_user = db.query(models.User).filter(models.User.username == user.username).first() 1
    if db_user.queue.current_position != -1:
        current_song = db.query(models.Song).\
            filter_by(id=db_user.queue.songs[db_user.queue.current_position]).\
            options(selectinload(models.Song.album)).\
            options(selectinload(models.Song.artists)).\
            first() 2
        add_like_attr(db_user, [current_song]) 4
        return current_song 3
    else:
        raise HTTPException(status_code=400, detail='Queue is empty')

```

Рисунок 26 — Метод получения текущего трека очереди

4.3.3 Плейлисты

Программный код CRUD класса плейлистов схож с классом очереди, т.к. он также реализован на основе массива, но в нем отсутствуют методы специфичные для очереди, но присутствуют проверки на принадлежности плейлиста пользователю и метод обновления изображения плейлиста, рисунок 27. Описание последовательности действий:

1. Проверка на принадлежность альбома пользователю;
2. Получение передаваемого изображения;
3. Валидация изображения и получение типа изображения;
4. Генерация uuid4 как имени нового изображения;
5. Сохранение полученного изображения;
6. Удаление предыдущего изображения (если оно не является стандартным).

```

async def update_playlist_image(
    self,
    db: Session,
    user: models.User,
    playlist_id: int,
    file: UploadFile=File(...)
):
    playlist = db.query(models.Playlist).filter_by(id=playlist_id).first()
    if playlist and playlist.user_id == user.id: 1
        contents = await file.read() 2
        image_type=imghdr.what(None,contents) 3
        if image_type:
            file.filename = f"{uuid.uuid4()}.{image_type}" 4
            path = f"{STATIC_PLAYLIST_PATH}/{file.filename}"
            with open(f"{main.APP_PATH}{path}", "wb") as f:
                f.write(contents) 5
            if playlist.cover_url != DEFAULT_PLAYLIST_COVER \
                and os.path.isfile(f"{main.APP_PATH}{playlist.cover_url}"):
                os.remove(f"{main.APP_PATH}{playlist.cover_url}") 6
            playlist.cover_url=path
            db.commit()
            db.flush()
            db.refresh(playlist)
            return playlist
        else:
            raise HTTPException(status_code=422, detail='Image validation error')
    else:
        raise HTTPException(status_code=404, detail='Playlist not found')

```

Рисунок 27 — Обновление изображения плейлиста

4.4 Стриминг

Программная реализация стриминга аудиофайлов приведена на рисунке 28. Описание реализации:

1. Функция-генератор, возвращающая указанные диапазоны байт файлового стрима;
2. Функция возвращает ответ класса `StreamingResponse`, содержащий диапазон байт указанный в `range` для указанного файла – `name`;
3. Если в запросе не указан диапазон байт, отдаем файл с его начала;
4. HTTP 1.1 поддерживает размерность указания диапазона запрашиваемого файла только в байтах (RFC 2616), указываем в заголовке `Accept-Ranges`;

5. Указываем возвращаемый диапазон байт и размер файла в заголовке Content-Range в формате “bytes начальный_байт-конечный_байт/размер_файла”;

6. Указываем тип возвращаемого контента, в нашем случае – audio/mpeg.

```
CONTENT_PART_SIZE = 100*1024

1
def part_generator(stream, part_size, start, total_size):
    bytes_read = 0
    stream.seek(start)
    while bytes_read < total_size:
        bytes_to_read = min(part_size, total_size - bytes_read)
        yield stream.read(bytes_to_read)
        bytes_read = bytes_read + bytes_to_read
    stream.close()

2
async def stream(name:str, range: Optional[str] = Header(None)):

    bytes_range = range or "bytes=0-" 3
    stream = open("/container/app/static/songs/" + name, 'rb')
    total_size = os.path.getsize("/container/app/static/songs/" + name)
    start_byte = int(bytes_range.split("=")[-1].split("-")[0])

    return StreamingResponse(
        part_generator(
            stream,
            start=start_byte,
            part_size=CONTENT_PART_SIZE,
            total_size=total_size
        ),
        headers={
            4
            "Accept-Ranges": "bytes",
            5
            "Content-Range": f"bytes {start_byte}-{start_byte+CONTENT_PART_SIZE}/{total_size}",
            6
            "Content-Type": "audio/mpeg",
        },
        status_code=206)
```

Рисунок 28 — Программная реализация стриминга контента

4.5 Поиск

Для настройки поисковой системы Elasticsearch выполнены следующие действия:

1. Установлен сервер Elasticsearch;
2. Созданы и сконфигурированы индексы для объектов, по которым осуществляется поиск;
3. Добавлены анализаторы к созданным индексам;
4. Перенесены необходимые данные в базу данных Elasticsearch в виде документов;
5. Созданы поисковые шаблоны запросов.

Т.к. Elasticsearch хранит все данные в собственной документно-ориентированной базе данных, то необходимо описать отображения хранящихся в ней объектов. Отображения созданы для объектов треков, альбомов и исполнителей, в них указаны только те атрибуты, по которым будет осуществляться поиск, рисунок 29.

<pre>mappings_songs = { "properties": { 1 "title": { "type": "text" }, "duration": { 2 "type": "date", 3 "format": "epoch_second" 4 }, "tags": { "type": "keyword" }, "album": { "type": "text" }, "artists": { "type": "text" } } }</pre>	<pre>mappings_albums = { "properties": { "title": { "type": "text" }, "release_date": { "type": "date", "format": "yyyy-MM-dd" }, "artists": { "type": "text" } } }</pre>	<pre>mappings_artists = { "properties": { "name": { "type": "text" } } }</pre>
--	---	--

Рисунок 29 — Конфигурация индексов

Описание параметров отображений:

1. Перечисление атрибутов, содержащихся в документе индекса;
2. Название атрибута;
3. Тип данных атрибута;
4. Формат хранящихся значений.

Для преобразования слов в необходимый для поиска вид (стемминг, удаление пробелов и знаков пунктуации и пр.) к каждому индексу применяется анализатор, конфигурация анализатора приведена на рисунке 30.

```
settings = {  
  "analysis": {  
    "filter": { 1  
      "ru_Ru": {  
        "type": "hunspell", 2  
        "locale": "ru_RU",  
        "dedup" : True  
      },  
      "ru_stemmer": {  
        "type": "stemmer",  
        "language": "russian"  
      }, ...  
    },  
    "analyzer": {  
      "default": {  
        "type": "custom",  
        "tokenizer": "standard",  
        "filter": [  
          "lowercase",  
          "ru_Ru",  
          "ru_stemmer", 3  
          "en_US",  
          "english_stemmer",  
          "english_possessive_stemmer"  
        ]  
      }  
    }  
  }  
}
```

Рисунок 30 — Конфигурация анализатора

Описание параметров конфигурации анализатора:

1. Определение фильтров и их имен;
2. Для выделения основ слов помимо стандартного стеммера «ru_stemmer» используется словарь hunspell. т.к. использование только алгоритмического подхода для русского языка показывало плохие результаты.;
3. Список всех используемых фильтров в анализаторе, фильтры английского аналогичны русскому языку.

Для работы с API Elasticsearch созданы шаблоны поисковых запросов, пример такого шаблона на рисунке 31.

```
"query": {  
  "bool": { 1  
    "filter": { 2  
      "range": {  
        "release_date": { 3  
          "gte": release_date_from,  
          "lte": release_date_to  
        }  
      }  
    },  
    "must": { 4  
      "match": {  
        "title": name  
      }  
    },  
    "should": { 5  
      "match": { "artists": name }  
    },  
  }  
}
```

Рисунок 31 — Шаблон поискового запроса для альбомов

Описание параметров поискового шаблона:

1. Тип сложного запроса `bool` – комбинирует `score`* для включенных в него условий;
2. Условие `filter` – указанный в нем запрос должен быть удовлетворен, но не влияет на `score`;
3. Запрос на фильтрацию по дате;
4. Условие `must` – указанный в нем запрос должен быть удовлетворен, влияет на `score`;
5. Условие `should` – указанный в нем запрос может быть удовлетворен, влияет на `score`.

* `score` – мера совпадения найденного объекта с запросом

Такая структура запроса гарантирует совпадение искомого названия альбома, а также позволяет дополнить поисковый запрос исполнителем этого альбома для более точного поиска. Подобный принцип также использовался для конфигурации поисковых запросов к трекам и исполнителям.

5 ТЕСТИРОВАНИЕ СИСТЕМЫ, РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

5.1 Автоматические тесты, Pytest

С использованием утилиты `pytest` написаны `unit` тесты для основных конечных точек API. Пример подобных тестов приведен на рисунке 32.

```
ACCESS_TOKEN = token.get_super_token()

client = AsyncClient(app=main.app, base_url="http://localhost")

@pytest.mark.anyio
async def test_read_user_me_200():
    headers = {"Authorization": "Bearer " + ACCESS_TOKEN}
    response = await client.get("/users/me", headers=headers)
    assert response.status_code == 200
    assert parse_obj_as(schemas.UserAll, response.json())

@pytest.mark.anyio
async def test_update_user_about_200():
    headers = {"Authorization": "Bearer " + ACCESS_TOKEN}
    json_about = {
        "name": "string",
        "surname": "string",
        "about": "string",
        "email": "string"
    }
    response = await client.post("/users/me", headers=headers, json=json_about)
    assert response.status_code == 200
    assert parse_obj_as(schemas.UserAbout, response.json())
    assert response.json() == json_about

@pytest.mark.anyio
async def test_read_user_me_artists_200():
    headers = {"Authorization": "Bearer " + ACCESS_TOKEN}
    response = await client.get("/users/me/artists", headers=headers)
    assert response.status_code == 200
    assert (parse_obj_as(List[schemas_req.ArtistLikeRequired], response.json()) or not response.json())
```

Рисунок 32 — Код тестирования конечных точек `/user/*`

Тестирование заключается в проверке правильности возвращаемых статус кодов ответа и проверке содержимого ответов для разных типов запросов. Запросы осуществляются через библиотеку `requests`, для проверки соответствия используется оператор `assert`.

Результаты проведения всех тестов приведены на рисунке 33.

```

root@8a6f388fd8d3:/container# python3 -m pytest /container/app/tests/unit/auth/ -w ignore::DeprecationWarning
===== test session starts =====
platform linux -- Python 3.9.10, pytest-7.1.2, pluggy-1.0.0
rootdir: /container
plugins: anyio-3.6.1
collected 66 items

app/tests/unit/auth/test_album_auth.py ..... [ 18%]
app/tests/unit/auth/test_artist_auth.py ..... [ 34%]
app/tests/unit/auth/test_home_auth.py ..... [ 48%]
app/tests/unit/auth/test_song_auth.py ..... [ 65%]
app/tests/unit/auth/test_tags.py .... [ 71%]
app/tests/unit/auth/test_token.py . [ 72%]
app/tests/unit/auth/test_users_me.py ..... [ 87%]
app/tests/unit/auth/test_users_username.py ..... [100%]

===== 66 passed in 11.77s =====
root@8a6f388fd8d3:/container#

```

Рисунок 33 — Результаты тестирования pytest

Все тесты успешно проходят проверки.

5.2 Ручное тестирование, тестирование поиска

Тестирование системы проводилось при помощи моделирования стандартных действий пользователя при работе с ней. Для выполнения запросов использовалась утилита `curl`, порядок действий:

1. Создание пользователя;
2. Вход в систему;
3. Обновление изображения пользователя;
4. Получение списка случайных треков жанра хип-хоп;
5. Оценка некоторых из полученных треков;
6. Получение рекомендаций к одному из альбомов этих треков;
7. Создание плейлиста и добавление нескольких песен;
8. Получение созданного плейлиста.

5.2.1 Создание пользователя

Выполнение запроса POST /users/, с указанием логина пользователя и пароля, рисунок 34.

```
curl -X 'POST' \
  'https://septerra.duckdns.org:44444/users/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "user123",
    "password": "strongpassword"
  }'
```

Рисунок 34 — Запрос на создание пользователя

Получен ответ об успешном создании пользователя с кодом HTTP 200, рисунок 35.

```
{
  "username": "user123",
  "image_url": "/static/images/user_avatars/default.png",
  "disabled": false,
  "name": null,
  "surname": null,
  "about": null,
  "email": null
}
```

Рисунок 35 — Ответ на запрос о создании пользователя

5.2.2 Вход в систему

Выполнение запроса POST /auth/token/, с указанием логина и пароля пользователя, рисунок 36.

```
curl -X 'POST' \
  'https://septerra.duckdns.org:44444/auth/token/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'grant_type=&username=user123&password=strongpassword&scope=&client_id=&client_secret='
```

Рисунок 36 — Запрос на получение токенов

Получен ответ содержащий токен доступа и обновления, рисунок 37. В дальнейших запросах токен доступа будет указываться в заголовке Authorization, как Authorization: Bearer {access_token}.

```
{
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyMTIzIiwidHlwZSI6ImFjY2VzcyIsImklIjoxMTEsImV4cCI6MTY1MzIxMTA4Nn0.oiXi6PcGYVCLEJH32JLRa5JrWY7rkjpDGzRGYKi12g",
  "refresh_token": "490c3c0aaeda4b12bf4c78a750c2c07f",
  "token_type": "bearer"
}
```

Рисунок 37 — Ответ на запрос получения токенов

5.2.3 Обновление изображения пользователя

Выполнение запроса POST /users/me/upload-image/, с прикрепленным изображением в виде бинарных данных как multipart/form-data, рисунок 38.

```
curl -X 'POST' \
  'https://septerra.duckdns.org:44444/users/me/upload-image/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyMTIzIiwidHlwZSI6ImFjY2VzcyIsImklIjoxMTEsImV4cCI6MTY1MzIxMTA4Nn0.oiXi6PcGYVCLEJH32JLRa5JrWY7rkjpDGzRGYKi12g' \
  -H 'Content-Type: multipart/form-data' \
  -F 'file=@002.jpg;type=image/jpeg'
```

Рисунок 38 — Запрос обновления пользовательского изображения

Получен ответ со сгенерированным путем к загруженной фотографии, рисунок 39.

```
{
  "id": 111,
  "username": "user123",
  "image_url": "/static/images/user_avatars/4a32c6e7-eb73-4a53-b9c2-f394f82b06cb.png"
}
```

Рисунок 39 — Ответ на запрос об обновлении изображения

5.2.4 Получение списка случайных треков

Выполнение запроса GET /home/random/songs/, с указанием тега hip-hop и количеством получаемых треков, рисунок 40.

```
curl -X 'GET' \
  'https://septerra.duckdns.org:44444/home/random/songs/?limit=10&tags=hip-hop' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWUiOiJ1c2VyMTIzIiwidHlwZSI6ImFjY2VzcyIsImklIjoxMTESImV4
cCI6MTY1MzIxMTA4Nn0.oiXi6PcGYVCLEJH32JLRa5JrWY7rkjpDGzRGYKi12g'
```

Рисунок 40 — Запрос случайных треков жанра hip-hop

Получен ответ, содержащий несколько объектов треков с необходимой информацией, рисунок 41.

```
[
  {
    "id": 105,
    "title": "Дождь 826",
    "duration": 123,
    "file_url": "/static/songs/2.mp3",
    "cover_url": "/static/images/song_covers/2.png",
    "liked": false,
    "album": {
      "id": 98,
      "title": "Let the Journey Begin EP 276",
      "release_date": "1994-10-06",
      "cover_url": "/static/images/album_covers/97.png",
      "liked": false
    },
    "artists": [
      {
        "id": 51,
        "name": "Independent Music Licensing Collective (IMLC) 711",
        "cover_url": "/static/images/artist_covers/71.png",
        "liked": false
      },
      ...
    ],
    "tags": [ ... {"id": 5, "name": "hip-hop"} ... ]
  },
  {
    "id": 191,
    "title": "GODANDRAM 925",
    "liked": false,
    ...
    "tags": [ ... {"id": 5, "name": "hip-hop"} ... ]
  },
  {
    "id": 191,
    "title": "GODANDRAM 925",
    "liked": false,
    ...
    "tags": [ ... {"id": 5, "name": "hip-hop"} ... ]
  },
  {
    "id": 389,
    "title": "Случайный вальс 7",
    "liked": false,
    ...
    "tags": [ ... {"id": 5, "name": "hip-hop"} ... ]
  }
]
```

Рисунок 41 — Ответ на запрос о получении случайных треков (усеченный)

5.2.5 Оценка треков

Выполнение запросов PUT /songs/105/like/ и PUT /songs/191/like/, рисунок 42 и 43 соответственно.

```
curl -X 'PUT' \
  'https://septerra.duckdns.org:44444/songs/105/like/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWUiOiJ1c2VyMTIzIiwidHlwZSI6ImFjY2VzcyIsImklIjoxMTEsImV4cCI6MTY1MzIxMTA4Nn0.oiXi6PcGYVCLEJH32JLRa5JrWY7rkjpDGzRGYKi12g'
```

Рисунок 42 — Запрос «лайка» трека с id равным 105

```
curl -X 'PUT' \
  'https://septerra.duckdns.org:44444/songs/191/like/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWUiOiJ1c2VyMTIzIiwidHlwZSI6ImFjY2VzcyIsImklIjoxMTEsImV4cCI6MTY1MzIxMTA4Nn0.oiXi6PcGYVCLEJH32JLRa5JrWY7rkjpDGzRGYKi12g'
```

Рисунок 43 — Запрос «лайка» трека с id равным 191

Получение ответов об успешном оценивании треков (true, т.к. треки до этого оценены не были), рисунок 44 и рисунок 45.

```
true
```

Рисунок 44 — Ответ на запрос «лайка» трека с id равным 105

```
true
```

Рисунок 45 — Ответ на запрос «лайка» трека с id равным 191

5.2.6 Получение рекомендаций к альбому

Выполнение запроса GET /albums/98/recommend/, с указанием количества получаемых рекомендаций к альбому, рисунок 46.

```
curl -X 'GET' \
  'https://septerra.duckdns.org:4444/albums/98/recommend/?skip=0&limit=2' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyMTIzIiwidHlwZSI6ImFjY2VzcyIsIm1kIjoxMTEsImV4cCI6MTY1MzIxMTA4Nn0.oiXi6PcGVVCLEJH32JLRa5JrWY7rkjpDGzRGYKi12g'
```

Рисунок 46 — Запрос получения рекомендаций к альбому с id равным 98

Получен ответ, содержащий два альбома, рисунок 47.

```
[
  {
    "id": 14,
    "title": "Float 495",
    "release_date": "1989-05-25",
    "cover_url": "/static/images/album_covers/13.png",
    "liked": false,
    ...
  },
  {
    "id": 39,
    "title": "Float 387",
    "release_date": "2002-04-03",
    "cover_url": "/static/images/album_covers/38.png",
    "liked": false,
    ...
  }
]
```

Рисунок 47 — Ответ на запрос о получении рекомендаций

5.2.7 Создание плейлиста, добавление песен

Выполнение запросов POST /users/me/playlists/, с указанием имени для нового плейлиста и POST /users/me/playlists/19/add-list/ с указанием списка треков, рисунок 48 и 49 соответственно.


```
curl -X 'POST' \
  'https://septerra.duckdns.org:44444/users/me/playlists/?name=Best%20playlist%20ever' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyMTIzIiwidHlwZSI6ImFjY2VzcyIsIm1kIjoxMTEsImV4cCI6MTY1MzIxMTA4Nn0.oiXi6PcGYVCLEJH32JLRa5JrWY7rkjpDGzRGYKi12g' \
  -d ''
```

Рисунок 48 — Запрос на создание плейлиста

```
curl -X 'POST' \
  'https://septerra.duckdns.org:44444/users/me/playlists/19/add-list/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyMTIzIiwidHlwZSI6ImFjY2VzcyIsIm1kIjoxMTEsImV4cCI6MTY1MzIxMTA4Nn0.oiXi6PcGYVCLEJH32JLRa5JrWY7rkjpDGzRGYKi12g' \
  -H 'Content-Type: application/json' \
  -d '[
  105,191,62
]'
```

Рисунок 49 — Запрос на добавление треков в плейлист

Получены следующие ответы, рисунок 50 и 51.

```
{
  "id": 19,
  "name": "Best playlist ever",
  "cover_url": "/static/images/playlist_covers/default.png"
}
```

Рисунок 50 — Ответ на запрос о создании плейлиста

```
true
```

Рисунок 51 — Ответ на запрос о добавлении треков в плейлист

5.2.8 Получение созданного плейлиста

Выполнение запроса GET /users/me/playlists/19/, рисунок 52.

```
curl -X 'GET' \
  'https://septerra.duckdns.org:44444/users/me/playlists/19/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlc2VyMTIzIiwidHlwZSI6ImFjY2VzcyIsImklkIjoxMTEsImV4cCI6MTY1MzIxMTA4Nn0.ioiXi6PcGYVCLEJH32JLRa5JrWY7rkjpDGzRGYKi12g'
```

Рисунок 52 — Запрос получения содержимого плейлиста

Получен ответ, содержащий добавленные ранее треки. Так как треки 105 и 191 были оценены, то поле liked в них поменялось на true, рисунок 53.

```
{
  "id": 19,
  "name": "Best playlist ever",
  "cover_url": "/static/images/playlist_covers/default.png",
  "songs": [
    {
      "id": 105,
      "title": "Дождь 826",
      "liked": true,
      ...
      "album": {...},
      "artists": [...],
      "tags": [...]
    },
    {
      "id": 191,
      "title": "GODANDRAM 925",
      ...
      "liked": true,
      "album": {...},
      "artists": [...],
      "tags": [...]
    },
    ...
  ]
}
```

Рисунок 53 — Ответ на запрос о получении содержимого плейлиста

Для проверки правильной работы поиска по основе слова и поиска по нескольким полям система протестирована с использованием двух запросов:

- Поиск трека с указанием фразы «Степи» (часть названия песни);
- Поиск трека с указанием фразы «Степи ДДТ» (часть названия песни и исполнитель).

Результат поисковых запросов приведен на рисунке 54.

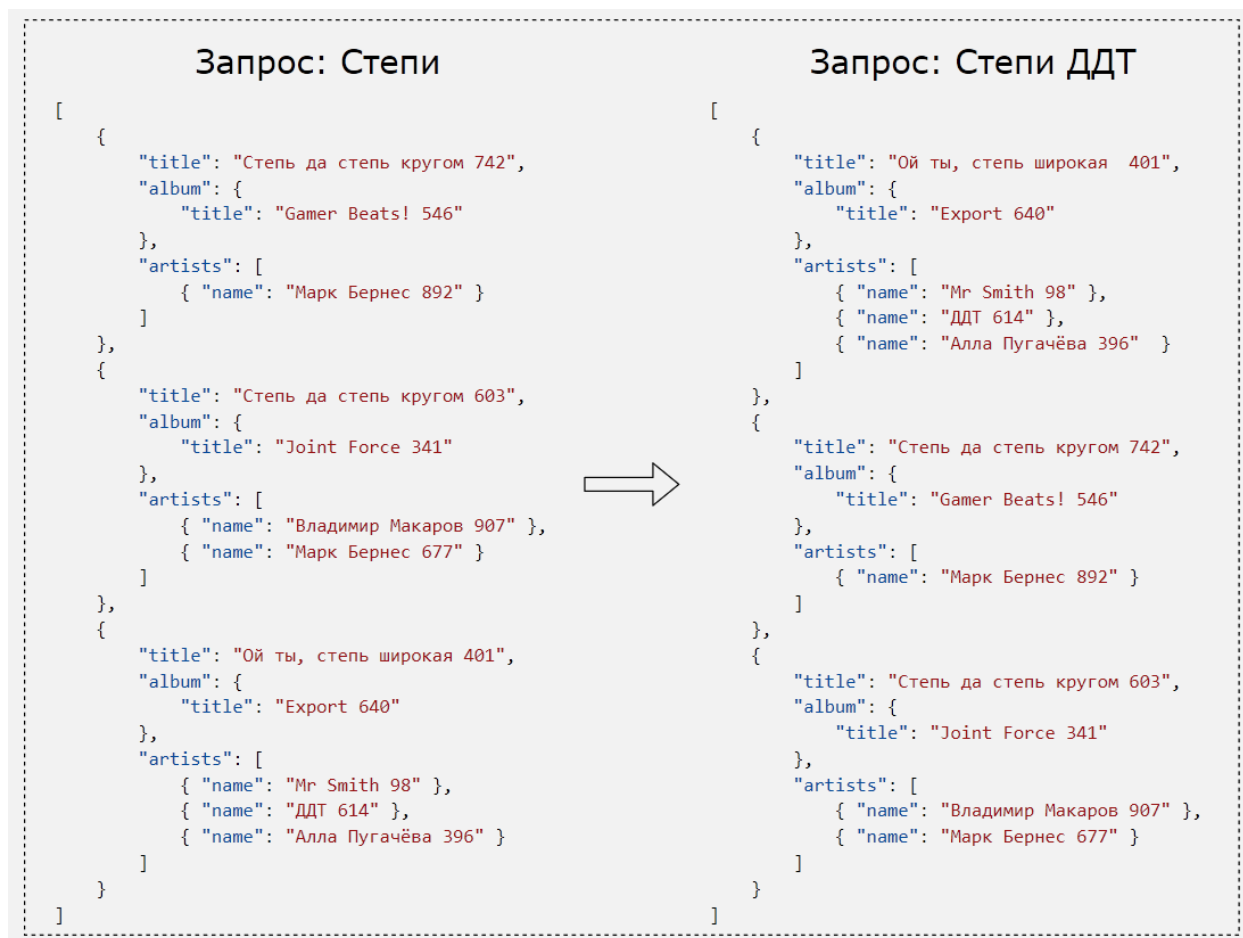


Рисунок 54 — Тестирование поисковых запросов, ответы усечены

В первом запросе найдены треки, содержащие слово «степь» или его производные, поиск по основе слова работает.

Во втором запросе найдены аналогичные треки, но изменился их порядок, чем выше трек в ответе — тем лучше результат совпадения. Кроме названия трека также указано имя одного из исполнителей, поэтому трек «Ой ты, степь широкая 401» находится выше в поисковом ответе, поиск по нескольким полям работает.

5.3 Руководство пользователя

В качестве дополнения к руководству пользователя (которым является разработчик системы) используется автоматически генерируемая документация SwaggerUI по спецификации OpenAPI. Вид документации представлен на рисунке 55.

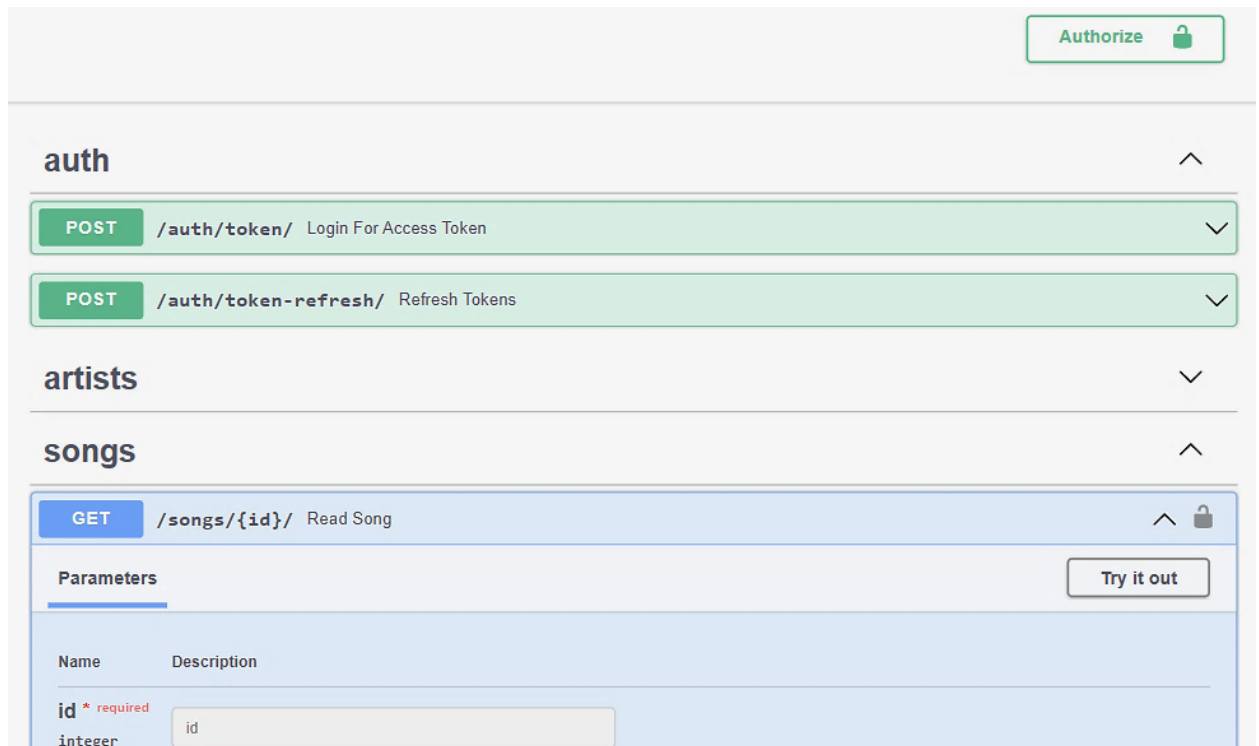


Рисунок 55 — SwaggerUI

SwaggerUI включает в себя описание всех конечных точек API, необходимость аутентификации, а также передаваемые параметры, их тип и стандартные значения, структуру передаваемого в теле запроса объекта при наличии. Для каждой конечной точки указываются возможные варианты ответов с указанием статус кода и схемы ответа, схемы ответа генерируются на основе Pydantic моделей, их использование позволяет проводить валидацию возвращаемых полей и гарантировать структуру возвращаемых ответов.

Пример схемы ответа для конечной точки получения трека по id представлен на рисунке 56.

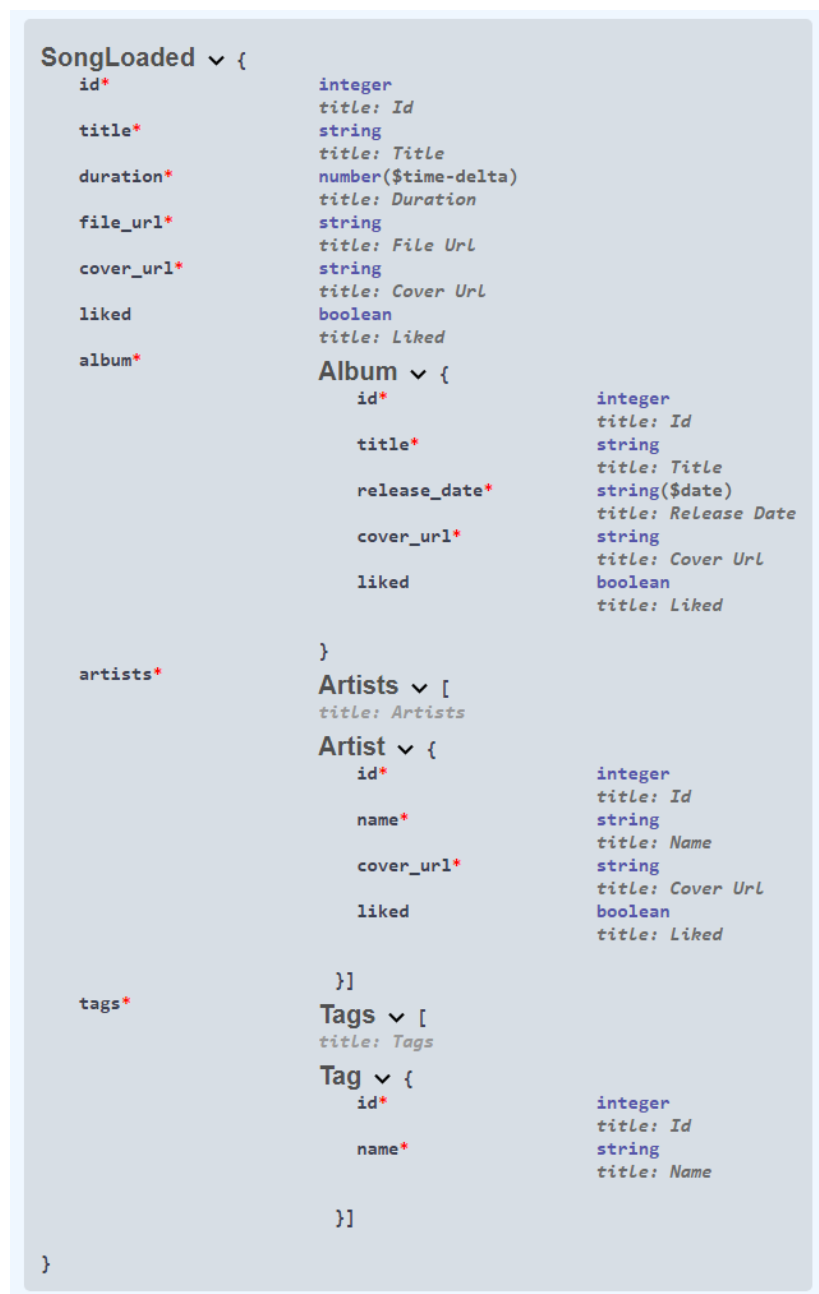


Рисунок 56 — Модель ответа SongLoaded

Аутентификация. Для использования любых функций системы пользователю необходимо пройти процедуру аутентификации. Для создания пользователя используется конечная точка POST /users/, принимающая следующий json объект

```

{
  "username": "your_username",
  "password": "your_password"
}

```

Имя пользователя может содержать только буквы латинского алфавита и цифры, длиной от 3 до 15 символов, имя пользователя должно быть уникальным.

Пароль пользователя может содержать любые символы, длиной от 3 до 15 символом.

Если хоть одно из условий не соблюдено, сервер возвращает ошибку валидации HTTP422 с описанием места ошибки (например, "Username can be 3 to 15 characters, no special characters").

Для аутентификации пользователя используется конечная точка /auth/token, принимающая в теле запроса следующий json объект

```
{
    "username": "your_username",
    "password": "your_password"
}
```

Если указанные реквизиты верны, сервер вернет токен доступа и токен обновления

```
{
    "access_token": "your_access_token",
    "refresh_token": "your_refresh_token"
}
```

Если указанные реквизиты неверны, сервер вернет ошибку HTTP 401 с описанием "Incorrect username or password".

Полученный access_token используется в заголовке Authorization, как Authorization: Bearer {your_access_token}, при любых запросах к защищенным ресурсам. Полученные токены доступа и обновления должны храниться на стороне клиента.

Полученный access_token действителен только 30 минут, для продолжительного пользования системой необходимо использовать механизм обновления токенов. Обновить токены можно используя конечную точку /auth/refresh-token/, принимающую refresh_token. Если refresh_token верен,

сервер вернет новую пару `access_token` и `refresh_token`, иначе ошибку HTTP 401 с описанием "Invalid refresh token".

Для автоматического обновления токенов можно использовать два варианта:

- Перед запросом к системе (или по таймеру) проверять время жизни токена, которое в открытом виде закодировано в сам токен, поле `exp`. В таком случае запрос на обновление токенов отправляется, когда время жизни `access_token` истекло;
- Обновлять токен при возврате ошибки истечения времени жизни токена (статус код – HTTP 401) при попытке обратиться к защищенной конечной точке API.

Пользователь. Для получения информации о пользователе (профиль пользователя) используется конечная точка `GET /users/me/`, возвращающая имя пользователя, `url` изображения пользователя, его имя и фамилию, поле «о себе» и `email`. Для обновления этой информации используется конечная точка `POST /users/me/`, принимающая `json` объект с полями `"name"`, `"surname"`, `"about"`, `"email"` (некоторые из полей могут отсутствовать).

Для обновления изображения пользователя используется конечная точка `POST /users/upload-image/`, принимающая изображения как `multipart/form-data`, в случае ошибки валидации изображения, возвращается ошибка HTTP 422 с описанием "Image validation error".

Треки, исполнители, альбомы. Для получения трека, исполнителя и альбома используются конечные точки `GET /songs/{id}`, `GET /artists/{id}`, `GET /albums/{id}` соответственно, для получения списка этих объектов используются точки `GET /albums/`, `GET /artists/`, `GET /albums/`, которые принимают параметры `limit` и `skip`, которые могут использоваться для ограничения количества получаемых объектов и получения диапазонов объектов. Для создания и

обновления объектов необходимо использовать соответствующие конечные точки и аккаунт администратора.

Плейлисты. Список всех плейлистов пользователя можно получить, используя конечную точку GET /users/me/playlists/, получить отдельный плейлист GET /users/me/playlists/{id}.

Для создания плейлиста используется конечная точка POST /users/me/playlists/ с указанием имени нового плейлиста. Для удаления плейлиста полностью используется DELETE /users/me/playlists/{id}/.

Для добавления одного трека, списка треков используются конечные точки POST /users/me/playlists/{id}/, POST /users/me/playlists/{id}/add-list/. Если отправленные id являются недействительными, сервер возвращает ошибку с описанием “Wrong id provided”.

Для обновления изображения плейлиста используется конечная точка POST /users/me/playlists/{id}/upload-image/, которая работает аналогично обновлению изображения пользователя.

Любые действия с плейлистами разрешены только для их владельцев, действия над чужими плейлистами приводят к ошибке HTTP 404 “Playlist not found”, также, как и действия над несуществующими плейлистами.

Очередь. Для получения текущей очереди треков используется конечная точка GET /users/me/queue/, текущего трека GET /users/me/queue/current/. Для получения следующего трека и перемещения позиции в очереди GET /users/me/queue/next/, предыдущего трека GET /users/me/queue/prev/.

Если очередь пуста, сервер возвращает пустой список треков и позицию в очереди равную -1. Если текущий трек отсутствует (очередь пуста), отсутствует предыдущий трек, отсутствует следующий трек, то сервер возвращает соответствующие ошибки: "Queue is empty", "No previous track", "No next track".

Принцип действия остальных конечных точек аналогичен таковым у плейлистов.

Поиск. Поиск разделен на три категории: поиск по альбомам, поиск по исполнителям, поиск по трекам, конечные точки: GET /search/albums/, GET /search/artists/, GET /search/songs/. Все точки принимают обязательный параметр name – имя по которому осуществляется поиск, параметр limit для ограничения количества результатов и дополнительные параметры фильтрации в зависимости от типа объекта.

Треки дополнительно фильтруются по передаваемому списку тегов (параметр tags), и длительности (параметры duration_from, duration_to).

Альбомы дополнительно фильтруются по дате выхода (параметры release_date_from и release_date_to).

Результаты поиска расположены в порядке убывания степени совпадения с запросом.

Главная страница (случайные рекомендации, последние релизы). Для получения случайного списка альбомов, треков и исполнителей используются конечные точки GET /random/albums/, GET /random/songs/, GET /random/artists/ с указанием параметра limit для ограничения количества результатов. Для треков дополнительно можно указать список тегов (параметр tags) среди которых будут выбираться случайные треки.

Для получения списка последних вышедших альбомов используется конечная точка GET /home/last-album-releases/ с указанием параметра limit.

ЗАКЛЮЧЕНИЕ

Основные результаты проведенной работы:

1. Проведена системная аналитика в области музыкальных сервисов и выделены их основные особенности на основе которых составлены бизнес-требования к системе.
2. Выбраны средства для разработки.
3. Выполнено архитектурное проектирование системы.
4. Выполнено функциональное проектирование системы.
5. К полученной системе написаны тесты, вручную протестированы отдельные ее части.

Результатом работы является созданное серверное приложение, предоставляющее возможности хранения, стриминга и классификации музыки, оценивание ее пользователями, рекомендации музыки на основе этих оценок.

СПИСОК ЛИТЕРАТУРЫ

1. Рейтинг самых популярных стриминговых сервисов для прослушивания музыки в России [Электронный ресурс]. - URL: <https://zebra-tv.ru/novosti/biznes/megafon-opredelil-reyting-samykh-populyarnykh-strimingovykh-servisov-dlya-proslushivaniya-muzyki-v-r/> (дата обращения: 23.04.2022).
2. Введение в REST API — RESTful веб-сервисы [Электронный ресурс]. – URL: <https://habr.com/ru/post/483202/> (дата обращения: 23.04.2022).
3. Python programming language [Электронный ресурс]. – URL: <https://www.python.org/> (дата обращения: 23.04.2022).
4. Flask's documentation [Электронный ресурс]. – URL: <https://flask.palletsprojects.com/en/2.1.x/> (дата обращения: 23.04.2022).
5. Django Web Framework [Электронный ресурс]. – URL: <https://www.djangoproject.com/> (дата обращения: 23.04.2022).
6. FastAPI Docs [Электронный ресурс]. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 27.04.2022).
7. Pydantic Data Validation [Электронный ресурс]. – URL: <https://pydantic-docs.helpmanual.io/> (дата обращения: 27.04.2022).
8. OpenAPI (спецификация). Википедия – свободная энциклопедия [Электронный ресурс]. – URL: [https://ru.wikipedia.org/wiki/OpenAPI_\(спецификация\)](https://ru.wikipedia.org/wiki/OpenAPI_(спецификация)) (дата обращения: 27.04.2022).
9. PostgreSQL: Open Source Relational Database [Электронный ресурс] – URL: <https://www.postgresql.org/> (дата обращения: 27.04.2022).
10. Сравнение PostgreSQL и MySQL [Электронный ресурс]. - URL: <https://sbercloud.ru/ru/warp/blog/mysql-vs-postgresql> (дата обращения: 23.04.2022).
11. MySQL. Реляционная база данных [Электронный ресурс]. – URL: <https://www.mysql.com/> (дата обращения: 27.04.2022).
12. SQLite. C-language library [Электронный ресурс]. – URL:

<https://www.sqlite.org/index.html> (дата обращения: 27.04.2022).

13. SQLite. Википедия – свободная энциклопедия [Электронный ресурс]. - URL: <https://ru.wikipedia.org/wiki/SQLite> (дата обращения: 27.04.2022).

14. Elastic. Index API. [Электронный ресурс]. – URL: https://www.elastic.co/guide/en/elasticsearch/reference/current/docs-index_.html (дата обращения: 02.05.2022).

15. PostgreSQL. Full Text Search [Электронный ресурс] – URL: <https://www.postgresql.org/docs/current/textsearch.html> (дата обращения: 02.05.2022).

16. Typesense. Open Source Search [Электронный ресурс]. – URL: <https://typesense.org/> (дата обращения: 02.05.2022).

17. Benchmarking Search Performance: Elasticsearch vs competitors [Электронный ресурс]. - URL: <https://blog.gigasearch.co/elasticsearch-against-competitors/> (дата обращения: 02.05.2022).

18. Hypercorn. ASGI web server [Электронный ресурс]. – URL: <https://pgjones.gitlab.io/hypercorn/> (дата обращения: 02.05.2022).

19. Uvicorn. An ASGI web server, for Python [Электронный ресурс]. – URL: <https://www.uvicorn.org/> (дата обращения: 02.05.2022).

20. Trio. Python library for async concurrency and I/O [Электронный ресурс]. – URL: <https://github.com/python-trio/trio> (дата обращения: 02.05.2022).

21. Uvloop. Fast asyncio event loop [Электронный ресурс] – URL: <https://github.com/MagicStack/uvloop> (дата обращения: 02.05.2022).

22. FastAPI. Deployment with uvicorn and gunicorn [Электронный ресурс]. – URL: <https://fastapi.tiangolo.com/deployment/server-workers/> (дата обращения: 01.05.2022).

23. SQLAlchemy 1.4 Documentation [Электронный ресурс]. – URL: <https://docs.sqlalchemy.org/en/14/orm/> (дата обращения: 01.05.2022).

24. Alembic. Database migration tool [Электронный ресурс] – URL: <https://alembic.sqlalchemy.org/en/latest/> (дата обращения: 01.05.2022).

25. Pytest 7.1 documentation [Электронный ресурс]. – URL:

<https://docs.pytest.org/en/7.1.x/> (дата обращения: 04.05.2022).

26. Docker. Official site [Электронный ресурс]. – URL: <https://www.docker.com/> (дата обращения: 04.05.2022).

27. RFC 7519. JSON Web Token (JWT) [Электронный ресурс]. - URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 04.05.2022).

28. RFC 2616. Hypertext Transfer Protocol -- HTTP/1.1 [Электронный ресурс]. - URL: <https://datatracker.ietf.org/doc/html/rfc2616> (дата обращения: 08.05.2022).

29. Can I use? Range Header [Электронный ресурс]. - URL: <https://caniuse.com/?search=Range%20header> (дата обращения: 11.05.2022).

30. ElasticSearch-py. Python Elasticsearch Client [Электронный ресурс]. - <https://elasticsearch-py.readthedocs.io/en/v8.2.2/> (дата обращения: 11.05.2022).

31. ElasticSearch Guide. Query DSL [Электронный ресурс]. – URL: <https://www.elastic.co/guide/en/elasticsearch/reference/current/query-dsl.html> (дата обращения: 11.05.2022).

ПРИЛОЖЕНИЯ

Приложение А. Описание конечных точек

Таблица А.1 — Описание конечных точек API

URN	Метод	Описание	Параметры запроса, тело запроса, часть URN	Доступность
1	2	3	4	5
/auth/token/	POST	Получение токена доступа и токена обновления	Тело запроса: <i>username: логин,</i> <i>password: пароль</i>	Неавторизованные пользователи
/auth/token-refresh/	POST	Обновления токена доступа и токена обновления	Тело запроса: <i>refresh_token: токен обновления</i>	Неавторизованные пользователи
/artists/	GET	Получение списка исполнителей	Параметры: <i>limit: количество необходимое получить,</i> <i>skip: сдвиг получаемых значений</i>	Неавторизованные пользователи
/artists/	POST	Создание исполнителя	Тело запроса: <i>name: имя исполнителя</i>	Администраторы
/artists/{id}/	PUT	Обновление исполнителя	Часть URN: <i>id: индивидуальный номер</i> Тело запроса: <i>name: имя исполнителя</i>	Администраторы
/artists/{id}/	DELETE	Удаление исполнителя	Часть URN: <i>id: индивидуальный номер</i>	Администраторы

Продолжение таблицы А.1

1	2	3	4	5
/artists/{id}/albums/	POST	Создание связи артист-альбом	Часть URN: <i>id: id исполнителя</i> Параметры: <i>album_id: id альбома</i>	Администраторы
/artists/{id}/albums/{id}/	DELETE	Удаление связи артист-альбом	Часть URN: <i>id: id исполнителя,</i> <i>album_id: id альбома</i>	Администраторы
/artists/{id}/update-image/	PUT	Обновление изображения исполнителя	Часть URN: <i>id: индивидуальный номер</i> Тело запроса (form-data): <i>Прикрепленное изображение</i>	Администраторы
/artists/{id}/like/	PUT	«Лайк» или «дизлайк» исполнителя по id	Часть URN: <i>id: индивидуальный номер</i>	Авторизованные пользователи
/songs/	GET	Получение списка треков	Параметры: <i>limit: количество</i> <i>необходимое получить,</i> <i>skip: сдвиг получаемых значений</i>	Неавторизованные пользователи
/songs/	POST	Создание трека	Тело запроса: <i>title: название трека,</i> <i>duration:</i> <i>продолжительность трека,</i> <i>album_id: id альбома</i>	Администраторы

Продолжение таблицы А.1

1	2	3	4	5
/songs/{id}/	PUT	Обновление трека	Часть URN: <i>id: индивидуальный номер</i> Тело запроса: <i>title: название трека,</i> <i>duration:</i> <i>продолжительность трека,</i> <i>album_id: id альбома</i>	Администраторы
/songs/{id}/	DELETE	Удаление трека	Часть URN: <i>id: индивидуальный номер</i>	Администраторы
/songs/{id}/artists/	POST	Создание связи трек-исполнитель	Часть URN: <i>id: id трека</i> Параметры: <i>artist_id: id исполнителя</i>	Администраторы
/songs/{id}/artists/{id}/	DELETE	Удаление связи трек-исполнитель	Часть URN: <i>id: id трека,</i> <i>artist_id: id исполнителя</i>	Администраторы
/songs/{id}/tags/	POST	Создание связи трек-тег	Часть URN: <i>id: id трека</i> Параметры: <i>tag_id: id тега</i>	Администраторы
/songs/{id}/tags/{tag_id}/	DELETE	Удаление связи трек-тег	Часть URN: <i>id: id трека,</i> <i>tag_id: id тега</i>	Администраторы

Продолжение таблицы А.1

1	2	3	4	5
/songs/{id}/update-image/	PUT	Обновление изображения трека	Часть URN: <i>id: индивидуальный номер</i> Тело запроса (form-data): <i>Прикрепленное изображение</i>	Администраторы
/songs/{id}/update-file/	PUT	Обновление файла трека	Часть URN: <i>id: индивидуальный номер</i> Тело запроса (form-data): <i>Прикрепленный файл трека</i>	Администраторы
/songs/{id}/	GET	Получение трека по id	Часть URN: <i>id: индивидуальный номер</i>	Неавторизованные пользователи
/songs/{id}/like/	PUT	«Лайк» или «дизлайк» трека по id	Часть URN: <i>id: индивидуальный номер</i>	Авторизованные пользователи
/albums/	GET	Получение списка альбомов	Параметры: <i>limit: количество необходимое получить,</i> <i>skip: сдвиг получаемых значений</i>	Неавторизованные пользователи
/albums/	POST	Создание альбома	Тело запроса: <i>title: название альбома,</i> <i>release_date: дата релиза</i>	Администраторы

Продолжение таблицы А.1

1	2	3	4	5
/albums/{id}/	PUT	Обновление альбома	Часть URN: <i>id – id альбома</i> Тело запроса: <i>title: название альбома,</i> <i>release_date: дата</i> <i>релиза</i>	Администратор ы
/albums/{id}/	DELETE	Удаление альбома	Часть URN: <i>id – id альбома</i>	Администратор ы
/albums/{id}/update-image/	PUT	Обновление изображения альбома	Часть URN: <i>id – индивидуальный</i> <i>номер</i> Тело запроса (form-data): <i>Прикрепленное</i> <i>изображение</i>	Администратор ы
/albums/{id}/	GET	Получение альбома по id	Часть URN: <i>id – индивидуальный</i> <i>номер</i>	Неавторизованн ые пользователи
/albums/{id}/like/	PUT	«Лайк» или «дизлайк» альбома по id	Часть URN: <i>id – индивидуальный</i> <i>номер</i>	Авторизованны е пользователи
/albums/{id}/recommend/	GET	Получить рекомендации к альбому с этим id	Часть URN: <i>id – индивидуальный</i> <i>номер</i> Параметры: <i>limit: количество</i> <i>необходимое получить,</i> <i>skip: сдвиг получаемых</i> <i>значений</i>	Неавторизованн ые пользователи

Продолжение таблицы А.1

1	2	3	4	5
/users/	POST	Создание пользователя	Тело запроса: <i>username: логин,</i> <i>password: пароль</i>	Неавторизованные пользователи
/users/me/	GET	Получение информации о пользователе (о себе)		Авторизованные пользователи
/users/me/	POST	Обновление информации о пользователе (о себе)		Авторизованные пользователи
/users/me/upload-image/	POST	Обновление изображения пользователя	Тело запроса (form-data): <i>Прикрепленное изображение</i>	Авторизованные пользователи
/users/me/artists/	GET	Получение понравившихся исполнителей (своих)		Авторизованные пользователи
/users/me/songs/	GET	Получение понравившихся треков (своих)		Авторизованные пользователи
/users/me/albums/	GET	Получение понравившихся альбомов (своих)		Авторизованные пользователи
/users/{username}/	GET	Получение информации о другом пользователе по его логину	Часть URN: <i>username: логин</i> <i>пользователя</i>	Неавторизованные пользователи

Продолжение таблицы А.1

1	2	3	4	5
/users/{username}/artists/	GET	Получение понравившихся другому пользователю исполнителей по его логину	Часть URN: <i>username: логин</i> <i>пользователя</i>	Неавторизованные пользователи
/users/{username}/songs/	GET	Получение понравившихся другому пользователю треков по его логину	Часть URN: <i>username: логин</i> <i>пользователя</i>	Неавторизованные пользователи
/users/{username}/albums/	GET	Получение понравившихся другому пользователю альбомов по его логину	Часть URN: <i>username: логин</i> <i>пользователя</i>	Неавторизованные пользователи
/users/me/queue/	GET	Получение текущего списка треков очереди		Авторизованные пользователи
/users/me/queue/current/	GET	Получение текущего трека в очереди		Авторизованные пользователи
/users/me/queue/add/	POST	Добавление трека в очередь по его id	Параметры: <i>song_id: id трека</i>	Авторизованные пользователи

Продолжение таблицы А.1

1	2	3	4	5
/users/me/queue/next/	PUT	Получение следующего трека и перемещение очереди вперед		Авторизованный пользователь
/users/me/queue/prev/	PUT	Получение предыдущего трека и перемещение очереди назад		Авторизованный пользователь
/users/me/queue/replace/	PUT	Замена очереди списком id треков	Тело запроса: <i>список id треков</i>	Авторизованный пользователь
/users/me/queue/delete/	DELETE	Удаления трека по указанной позиции из очереди	Параметры: <i>position: позиция в очереди</i>	Авторизованный пользователь
/users/me/queue/clear/	DELETE	Очистить очередь		Авторизованный пользователь
/users/me/queue/set/	PUT	Установить позицию в очереди на указанную	Параметры: <i>position: позиция в очереди</i>	Авторизованный пользователь
/users/me/queue/add-list/	PUT	Добавить список треков в конец очереди	Тело запроса: <i>список id треков</i>	Авторизованный пользователь

Продолжение таблицы А.1

1	2	3	4	5
/users/me/playlists/	GET	Получение плейлистов (своих)		Авторизованные пользователи
/users/me/playlists/	POST	Создание плейлиста с указанным именем	Параметры: <i>name: имя плейлиста</i>	Авторизованные пользователи
/users/me/playlists/{id}/	GET	Получение плейлиста с указанным id	Часть URN: <i>id: id плейлиста</i>	Авторизованные пользователи
/users/me/playlists/{id}/	DELETE	Удаление плейлиста с указанным id	Часть URN: <i>id: id плейлиста</i>	Авторизованные пользователи
/users/me/playlists/{id}/upload-image	POST	Обновление обложки плейлиста	Часть URN: <i>id: id плейлиста</i> Тело запроса (form-data): <i>Прикрепленное изображение</i>	Авторизованные пользователи
/users/me/playlists/{id}/add/	POST	Добавление трека в указанный плейлист по их id	Часть URN: <i>id: id плейлиста</i> Параметры: <i>song_id: id трека</i>	Авторизованные пользователи
/users/me/playlists/{id}/add-list/	POST	Добавление списка треков в указанный плейлист	Часть URN: <i>id: id плейлиста</i> Тело запроса: <i>список id треков</i>	Авторизованные пользователи
/users/me/playlists/{id}/clear/	DELETE	Удаление всех треков указанного по id плейлиста	Часть URN: <i>id: id плейлиста</i>	Авторизованные пользователи

Продолжение таблицы А.1

1	2	3	4	5
/users/me/playlists/{id}/delete/	DELETE	Удаление трека по указанной позиции из плейлиста	Часть URN: <i>id: id плейлиста</i> Параметры: <i>position: позиция в плейлисте</i>	Авторизованные пользователи
/search/artists/	GET	Поиск среди артистов по указанной фразе	Параметры: <i>limit: ограничение количества результатов,</i> <i>name: поисковая фраза</i>	Неавторизованные пользователи
/search/albums/	GET	Поиск среди альбомов по указанной фразе	Параметры: <i>limit: ограничение количества результатов,</i> <i>name: поисковая фраза</i>	Неавторизованные пользователи
/search/songs/	GET	Поиск среди треков по указанной фразе	Параметры: <i>limit: ограничение количества результатов,</i> <i>name: поисковая фраза</i>	Неавторизованные пользователи
/tags/	GET	Получение списка имеющихся жанровых тегов		Неавторизованные пользователи
/tags/	POST	Добавление тега	Тело запроса: <i>name: название жанра</i>	Администраторы
/tags/{id}/	DELETE	Удаление тега	Часть URN: <i>id: id тега</i>	Администраторы

Продолжение таблицы А.1

1	2	3	4	5
/tags/songs/	GET	Получение песен по указанным жанровым тегам	Параметры: <i>limit: количество необходимое получить, skip: сдвиг получаемых значений tags: список тегов</i>	Неавторизованные пользователи
/home/random/albums/	GET	Получение случайных альбомов	Параметры: <i>limit: количество необходимое получить</i>	Неавторизованные пользователи
/home/random/songs/	GET	Получение случайных песен по указанному списку тегов	Параметры: <i>limit: количество необходимое получить tags: список тегов</i>	Неавторизованные пользователи
/home/random/artists/	GET	Получение случайных исполнителей	Параметры: <i>limit: количество необходимое получить</i>	Неавторизованные пользователи